

第二篇 计算机系统结构：从晶体管到超级大脑

你正坐在考场里，卷子发下来了。一道题写着："某程序可并行部分占80%，使用4个处理器，求加速比。"你盯着这道题，大脑一片空白。

BUT WAIT — 如果你读完这一篇，这道题你30秒就能搞定。

计算机系统结构，听起来又硬又干，像一块啃不动的钢板？别被骗了。它其实是计算机科学中最"性感"的部分——你想知道为什么你的iPhone能同时播放音乐、下载文件、刷新微信吗？你想知道为什么打开一个100MB的Excel比打开一个1KB的txt慢那么多吗？你想知道超级计算机为什么能预测天气，而你电脑玩个3A游戏就发烫吗？

答案全在这一篇里。

老张（我们那位头发花白但眼神犀利的架构师）说过一句名言："软件是灵魂，但系统结构是骨架。没有骨架，灵魂就是一滩烂泥。"

来，跟**小明**一起，钻进计算机的最底层，看看那些0和1是怎么跑起来的。

第1章. 计算机系统结构导论：站在巨人的肩膀上

老张放下咖啡杯，看着一脸迷茫的**小明**。

"小明，你说说看，什么是计算机？"

"呃.....就是那个.....有屏幕、有键盘、能上网的东西？"

"那是'电脑'。我问的是'计算机系统结构'研究的是什么。"

小明挠了挠头。老张笑了笑："别急，这一章学完，你不仅能回答这个问题，还能在考场上秒杀Flynn分类法和Amdahl定律。"

计算机系统的"千层饼"

你有没有想过，你敲下一行`print("Hello")`的时候，计算机到底经历了什么？

[Diagram: 一个七层蛋糕，从下到上标注：数字逻辑层、微程序层、机器语言层、操作系统层、汇编语言层、高级语言层、应用层。每一层之间有箭头指向上下。]

这就是**计算机系统的多级层次结构**。从最底层的晶体管开关（数字逻辑层）到最顶层的微信、浏览器（应用层），中间有好多"翻译官"在工作。

GEEK NOTE

这个分层思想是计算机科学中最伟大的抽象之一。每一层只关心自己的事情，不用管下面怎么实现的。就像你开车不用知道内燃机怎么工作——踩油门，车就走。

关键概念来了：**系统结构（Architecture）、组成（Organization）和实现（Implementation）** 是三个不同的东西。

层次	管什么	举个例子
系统结构	程序员看到的"抽象界面"——指令集、数据类型、寻址方式	x86指令集、IEEE 754浮点
组成	怎么用硬件实现这个界面——数据通路、控制单元、Cache	ALU设计、流水线
实现	具体的物理技术——用什么材料、什么工艺	7nm vs 5nm工艺、CMOS

老张敲了敲白板："同一个指令集（比如x86），可以用不同的组成去实现（Intel和AMD内部完全不一样），更可以用不同的实现技术（14nm、7nm）。这三层各自独立演进。"

WATCH IT!

很多初学者把"系统结构"和"组成"混为一谈。考试最爱考：**指令集属于系统结构，Cache大小属于组成，芯片工艺属于实现**。记牢这三个例子！

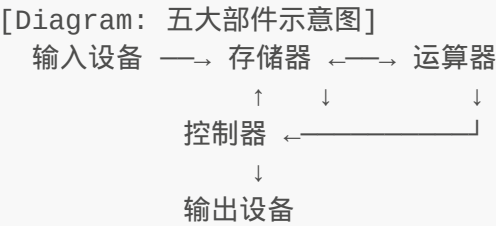
小明皱着眉："那冯·诺依曼结构呢？考试肯定考这个吧？"

冯·诺依曼：一个改变世界的"饭局"

1944年，冯·诺依曼在火车站月台等车时，听到了ENIAC计算机的消息。他没有回家，而是直接加入了项目。然后他写了一份报告，定义了现代计算机的基本框架。

这份报告的含金量？**直到今天，你用的每一台电脑、每一部手机，都还在用这个框架。**

冯·诺依曼结构的五大部件：



- 1. **运算器（ALU）**：做算术和逻辑运算
- 2. **控制器（CU）**：指挥整个乐队，取指令、译码、执行
- 3. **存储器**：存程序和数据（程序和数据放在同一个存储器里！）
- 4. **输入设备**：键盘、鼠标等
- 5. **输出设备**：显示器、打印机等

核心思想：**存储程序**。程序像数据一样，以二进制形式存放在存储器中，一条一条取出来执行。

在冯·诺依曼之前，程序是靠插拔导线来"写"的。改一次程序可能要重新接几百根线。冯·诺依曼说：把程序存在存储器里，想改程序就改存储内容。这个人简直是程序员的救世主。

SHARPEN YOUR PENCIL

请填空，补全冯·诺依曼结构的五大部件及其核心思想。

1. 冯·诺依曼结构的五大部件是：、、、 ____。
2. 冯·诺依曼结构的核心思想是 ____。
3. 在冯·诺依曼结构中，程序和数据存放方式是 ____。

(答案在下一页。)

SHARPEN YOUR PENCIL SOLUTION

1. 运算器、控制器、存储器、输入设备、输出设备
2. 存储程序（程序以二进制形式存储在存储器中，逐条取出执行）
3. 程序和数据存放在同一个存储器中，都按地址访问

Flynn分类法：四种人格的计算机

老张又发问了："小明，你说Intel Core i7是哪种计算机？"

"呃.....冯·诺依曼？"

"那是结构，不是分类。1966年，Mike Flynn提出了一个经典的分类法，考察两个维度：**指令流**和**数据流**。"

所谓指令流，就是机器执行的指令序列；数据流，就是指令操作的数据序列。每条路都可以是"单个（Single）"或"多个（Multiple）"。2×2=4，四种类型：

[Diagram: 2×2矩阵]

	单数据流 (SD)	多数据流 (MD)
单指令流 (SI)	SISD 传统单核CPU	SIMD 向量处理机、GPU
多指令流 (MI)	MISD 理论上存在， 实际极少见	MIMD 多核CPU、多处理机

SISD：你只有一个厨师，只有一口锅，一次炒一个菜。这就是传统单核CPU。

SIMD：你有一个厨师，但有很多口锅。厨师喊一声"放盐"，所有锅同时放盐。GPU用的就是这种思路——一个指令，多条数据同时处理。

MISD：很多厨师排队，每个人在前一个人做的菜上再加点料。流水线有点像，但严格意义上的MISD很少见。
考试一般不考这个。

MIMD：很多厨师，很多口锅，各炒各的。这就是现在的多核CPU、多处理机系统。

THERE ARE NO DUMB QUESTIONS

Q: GPU是SIMD，那我的CPU也是多核的，它算什么？ A: 好问题！你的多核CPU整体是MIMD（每个核执行不同的程序），但每个核内部的SSE/AVX指令集又是SIMD（一条指令处理128/256/512位数据，相当于同时做多个运算）。这就是为什么现代处理器是混合型的——宏观MIMD，微观SIMD。

Q: Flynn分类法有局限吗？A: 当然有。比如它只看指令流和数据流的数量，完全不关心存储器的组织方式、互连方式等。但它仍然是考试和教科书中最重要的分类法，因为它足够简洁和直观。

BRAIN POWER

假设你有一台普通笔记本电脑（4核Intel i5）和一台NVIDIA RTX 4090显卡。按Flynn分类法，笔记本CPU整体属于哪种？GPU核心又属于哪种？更深一层想：CPU每个核里的AVX-512指令集又算什么？

（提示：一个设备可以同时属于多个分类层级。）

SHARPEN YOUR PENCIL

将下列计算机系统归类到相应的Flynn分类：

1. 1990年的单核Intel 486
2. NVIDIA RTX 4090 GPU 的 CUDA Core 阵列
3. 现代4核Intel i5笔记本电脑（整体视角）
4. 向量机Cray-1执行向量加法时
5. 存在多个独立CPU的大型机，每个CPU执行不同用户的程序

SHARPEN YOUR PENCIL SOLUTION

1. SISD — 单核单指令单数据流
2. SIMD — GPU核心阵列执行相同指令处理不同数据
3. MIMD — 4个核各自执行不同的指令处理不同的数据
4. SIMD — 一条向量加法指令同时操作多个数据元素
5. MIMD — 多个CPU独立执行不同用户的程序，多条指令流多数据流

从独木桥到高速公路：并行性的演进

计算机刚诞生时，执行一条指令：取指→译码→执行→写回，做完一条再做下一条。这叫**顺序执行**。慢得令人发指。

然后有人想：取完这条指令后，别等执行完，先取下一条指令呗！这叫**先行控制**（Lookahead）——在不改变程序语义的前提下，提前做能做的事。

再然后，聪明人又想了：把一条指令的执行拆成多个小步骤，像工厂流水线一样。时钟一拍，这些步骤同时进行——第一条指令在做加法，第二条指令在译码，第三条指令在取指。这就是**流水线**（Pipeline）。效率暴增！

再进一步，多个处理单元同时干活，这就是**并行处理**（Parallel Processing）。

[Diagram: 并行性发展路线图]

顺序执行 → 先行控制 → 流水线 → 多处理机并行
（单车道）（带超车道的单车道）（工厂流水线）（多条高速并行）

软件说：我要搬家！——可移植性问题

小明问："为什么我在Windows上写的程序，在Mac上就跑不了？"

老张叹了口气："因为指令集不同。Windows用的是x86/x64，Mac现在用ARM（Apple Silicon）。不同的处理器说不同的'语言'。"

这引出了一个关键问题：**软件可移植性**。

业界想了很多招：

系列机：同一个厂商，不同型号用相同的指令集。Intel的Core i3、i5、i7、i9都兼容x86/x64指令集。程序不用改就能跑。

兼容机：不同厂商，但用兼容的指令集。AMD的处理器能跑Intel上写的程序。

模拟（Simulation）：用软件模拟另一个指令集。在x86上用虚拟机跑ARM程序——慢，但能用。

仿真（Emulation）：用硬件（微程序）来实现另一个指令集。比纯软件模拟快。

WATCH IT!

考试经常出的辨析题：**模拟是用软件实现，仿真是用硬件/固件实现**。模拟慢但灵活，仿真快但成本高。记住：Simulation=Software，Emulation=firmware/hardware。两个E对两个E！

拍个X光：怎么给计算机测"体能"？

老张说："性能评价是考试的计算题重灾区。打起精神来！"

CPU性能公式

最核心的公式：

$$\text{CPU执行时间} = \text{指令数} \times \text{CPI} \times \text{时钟周期}$$

其中 **CPI**（Cycles Per Instruction）是每条指令平均需要的时钟周期数。

反过来：**MIPS**（Million Instructions Per Second）= 时钟频率（MHz）/ CPI

MFLOPS（Million Floating-point Operations Per Second）：每秒百万次浮点运算。专用于科学计算领域。

GEEK NOTE

MIPS有一个著名的缺陷：不同指令集架构下，MIPS值不可直接比较。RISC机器一条指令做的事可能比CISC少，所以RISC的MIPS看起来高，但不代表实际性能强。这就是所谓的"MIPS陷阱"。

让我们算一道题把概念串起来：

例题：某CPU主频500MHz，运行某程序需要5秒，该程序共有10亿条指令。求：(1) CPI (2) MIPS

一步步来：

(1) 每个时钟周期的时间 = $1 / 500\text{MHz} = 1 / (500 \times 10^6) = 2 \times 10^{-9}$ 秒 = 2ns

(2) CPU执行时间 = 指令数 × CPI × 时钟周期

$$5 = 10^9 \times \text{CPI} \times 2 \times 10^{-9}$$

$$5 = \text{CPI} \times 2$$

$$\text{CPI} = 2.5$$

$$(3) \text{ MIPS} = \text{时钟频率(MHz)} / \text{CPI} = 500 / 2.5 = \mathbf{200 \text{ MIPS}}$$

$$\text{或者: MIPS} = \text{指令数} / (\text{执行时间} \times 10^6) = 10^9 / (5 \times 10^6) = 200$$

Amdahl定律：优化有上限

你升级了一半代码，性能能提升多少？Amdahl说：没那么简单。

$$\text{加速比} = \frac{1}{(1-f) + f/S}$$

其中 f = 可改进部分占总时间的比例， S = 该部分的加速倍数。

例题：某程序80%的时间用于可并行的计算，若使用4个处理器并行这部分，整个程序的加速比是多少？

$$\text{解: } f = 0.8, S = 4$$

$$\text{加速比} = \frac{1}{(1-0.8) + 0.8/4} = \frac{1}{0.2 + 0.2} = \frac{1}{0.4} = 2.5$$

关键结论：即使你把80%的代码用4个核并行（理论上加速4倍），整体程序加速比也只有2.5。那20%的串行部分就像瓶颈的瓶口——不管瓶身多宽，水流只能从瓶口走。

极限情况：如果处理器无限多（ $S \rightarrow \infty$ ），加速比 = $1/(1-f) = 1/0.2 = 5$ 。这就是加速比的**理论上限**。

SHARPEN YOUR PENCIL

计算下列场景的加速比：

1. 某程序可并行部分占60%，使用8个处理器的加速比是多少？
2. 某程序90%可并行，若加速比要达到5，至少需要多少个处理器？
3. 某程序在单核上运行时间100秒，其中80秒可并行。使用16核的加速比是多少？理论上限是多少？

（做完再看答案，别偷懒！）

SHARPEN YOUR PENCIL SOLUTION

1. $f=0.6, S=8$: 加速比 = $1/(0.4 + 0.6/8) = 1/(0.4+0.075) = 1/0.475 \approx \mathbf{2.11}$
2. 加速比=5, $f=0.9$, 求 S : $5 = 1/(0.1 + 0.9/S) \rightarrow 0.1 + 0.9/S = 0.2 \rightarrow 0.9/S = 0.1 \rightarrow S = \mathbf{9个处理器}$
3. $f=80/100=0.8, S=16$: 加速比 = $1/(0.2 + 0.8/16) = 1/(0.2+0.05) = 1/0.25 = \mathbf{4}$ 。理论上限 = $1/0.2 = \mathbf{5}$ 。

基准程序与SPEC

性能评测不能靠猜，要用**基准程序**（Benchmark）来实测。SPEC（Standard Performance Evaluation Corporation）是最权威的基准测试组织。

SPEC CPU2006、SPEC CPU2017等是业界标准测试套件。考试了解一下就好。

BE THE COMPUTER

下面是一段程序在某个CPU上的执行过程。请你扮演CPU，追踪每条指令的执行，然后计算总CPI和MIPS。

某CPU主频1GHz，程序指令序列及各类指令的CPI如下：

指令类型	出现次数	CPI
取数/存数	300	2
整数运算	500	1
分支指令	200	3
浮点运算	100	5

程序总指令数 = 1100 条

请计算：

1. 总时钟周期数
2. 平均CPI
3. 总执行时间
4. MIPS值

BE THE COMPUTER SOLUTION

1. 总时钟周期数 = $300 \times 2 + 500 \times 1 + 200 \times 3 + 100 \times 5 = 600 + 500 + 600 + 500 = 2200$ 个周期
2. 平均CPI = 总时钟周期 / 总指令数 = $2200 / 1100 = 2.0$
3. 时钟周期 = $1/1\text{GHz} = 1\text{ns} = 10^{-9}\text{s}$ 总执行时间 = $2200 \times 10^{-9} = 2.2 \times 10^{-6} = 2.2$ 微秒
4. MIPS = 时钟频率(MHz)/CPI = $1000/2.0 = 500$ (MIPS) 或：MIPS = $1100/(2.2 \times 10^{-6} \times 10^6) = 500$

BULLET POINTS（第1章）

- 计算机系统是**多级层次结构**：从数字逻辑到应用层，每一层屏蔽下层细节。
- **系统结构、组成、实现**三者不同：指令集是结构，数据通路是组成，工艺是实现。
- **冯·诺依曼结构**五大部件：运算器、控制器、存储器、输入设备、输出设备，核心是**存储程序**。
- **Flynn分类法**按指令流×数据流分为SISD、SIMD、MISD、MIMD。GPU是SIMD，多核CPU是MIMD。
- **CPU执行时间 = 指令数 × CPI × 时钟周期**。这是性能公式的灵魂。
- **MIPS = 时钟频率(MHz) / CPI**。注意MIPS的跨平台比较陷阱。
- **Amdahl定律**：加速比 = $1 / ((1-f) + f/S)$ 。串行部分是性能瓶颈的根源。
- 软件可移植性：**系列机**同指令集不同实现，**模拟**用软件，**仿真**用硬件/固件。
- 并行性发展路径：顺序执行 → 先行控制 → 流水线 → 并行处理。
- **SPEC基准程序**是业界标准性能评测工具。

第2章. 数据表示与指令系统：0和1的七十二变

小明在笔记本上写下了32个0和1，然后绝望地看着老张。

"十进制数 -15.75 怎么变成这么长一串？还有，为什么同样是32位二进制，有些代表整数，有些代表小数？CPU怎么知道谁是谁？"

老张嘏了一口茶："问得好。CPU其实**不知道**。同样32位01111111100000000000000000000000，在你眼里可能是整数2139095040，也可能是浮点数 $+\infty$ 。**区别在于你用什么指令去解释它。**"

这一章，我们就把这些0和1的妆容一层层卸掉。

定点数：当小数点是固定的

"定点"的意思是小数点位置固定。通常约定小数点在最右边（纯整数）或最左边（纯小数）。

原码：最高位是符号位（0正1负），其余位表示绝对值。

- $+5 = 0\ 0000101$ （假设8位）
- $-5 = 1\ 0000101$

问题？两个零： $+0 = 0\ 0000000$, $-0 = 1\ 0000000$ 。而且加减法时符号位要单独处理，电路复杂。

反码：正数反码=原码；负数反码=符号位不变，其余位取反。

- $+5 = 0\ 0000101$
- $-5 = 1\ 1111010$

反码的0还是有两个： $+0 = 00000000$, $-0 = 11111111$ 。但反码是通往补码的桥梁。

补码：正数补码=原码；负数补码=反码+1。

- $+5 = 0\ 0000101$
- $-5 = 1\ 1111011$

补码的伟大之处：0只有一个（00000000），且加减法统一处理！ $[A]_{\text{补}} + [B]_{\text{补}} = [A+B]_{\text{补}}$ ，符号位直接参与运算。所以**现代计算机几乎都用补码**。

移码：在补码基础上，符号位取反。也就是真值 + 偏移量（通常 $2^{(n-1)}$ ）。移码的好处是：可以直接比较两个数的大小——就跟无符号数一样。IEEE 754浮点数的阶码用的就是移码。

GEEK NOTE

补码为什么" $A-B = A + (-B)$ 的补码"就行了？因为补码把减法变成了加法。在模 2^n 的系统中， x 的补码就是 $2^n - x$ （当 $x < 0$ 时）。所以减B等价于加 $(2^n - B)$ ，绕了一圈又回到原点。这个"模运算"的概念是理解补码的核心。

溢出的判断

两个正数相加得负数？两个负数相加得正数？这就是**溢出**。

判断溢出的方法：

- **单符号位法**：进位位 C_s （最高数值位进位）和 C_p （符号位进位），溢出 = $C_s \oplus C_p$ （异或）
- **双符号位法**：用两位符号位。00正，11负。结果01=正溢出，10=负溢出。正常结果00或11。

WATCH IT!

Cs和Cp异或为1时溢出？如果两个正数相加，符号位都是0，Cs=1（有向符号位的进位）时，Cp=0， $1 \oplus 0 = 1$ ，溢出。记住了吗？正+正不应变负！

浮点数：当小数点是"浮"的

如果一个数可以是3.14159，也可以是299792458，也可以是0.0000000001，你不可能为每个数都固定小数点位置。所以要用**科学计数法**。

十进制：299792458 = 2.99792458×10^8

二进制也一样：小数点在"浮动"，用**指数**跟踪它。

IEEE 754标准

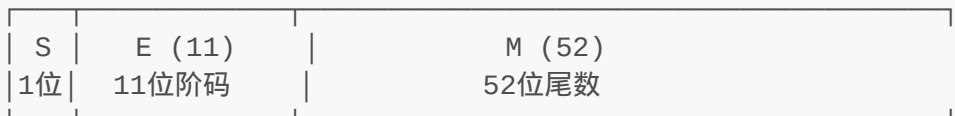
这是1985年制定的浮点数标准，现在全球通用。

单精度（32位）：

[Diagram: 32位单精度浮点数格式]



双精度（64位）：



S（符号位）：0正1负 E（阶码）：用**移码**表示，偏移量 = $2^{(k-1)} - 1$ （k是阶码位数）。单精度偏移127，双精度偏移1023。M（尾数）：规格化后小数部分（省去最高位的隐含1）

真值 = $(-1)^S \times (1.M) \times 2^{(E - \text{偏移量})}$

SHARPEN YOUR PENCIL

请将十进制数 -15.75 转换为IEEE 754单精度浮点数，写出完整步骤。

（给你一点提示：先不管符号，转成二进制，然后规格化，再确定阶码和尾数。）

SHARPEN YOUR PENCIL SOLUTION

步骤：

1. $15 = 1111_B$, $0.75 = 0.11_B \rightarrow 15.75 = 1111.11_B$

2. 规格化： $1111.11 = 1.1111 \times 2^3$

3. 符号位 $S = 1$ (负数)
4. 阶码 $E = 3 + 127(\text{偏移}) = 130 = 10000010\text{B}$ (8位)
5. 尾数 $M = 11111000000000000000000$ (23位, 去掉隐含1)

结果：

```
S = 1
E = 10000010
M = 11111000000000000000000

合起来：1 10000010 11111000000000000000000
即：1100 0001 0111 1100 0000 0000 0000 0000 (十六进制：C17C0000H)
```

IEEE 754的特殊值

E的取值	M的取值	代表的值
全0	全0	± 0
全0	非0	非规格化数
全1	全0	$\pm \infty$ (无穷大)
全1	非0	NaN (Not a Number, 如0/0)
其他	任意	规格化浮点数

WATCH IT!

阶码全0时, 隐含位不再是1而是0! 也就是说非规格化数 $= (-1)^S \times (0.M) \times 2^{(1-\text{偏移})}$ 。这种设计可以让浮点数平滑过渡到0, 避免在极小的数附近出现"下溢到0"的断层。太聪明了!

BRAIN POWER

IEEE 754单精度能表示的最大规格化正数是多少? 最小规格化正数呢? 最小非规格化正数呢?

(提示: 最大规格化正数: $E=11111110$, M 全1。最小规格化正数: $E=00000001$, M 全0。最小非规格化正数: $E=00000000$, M 只有最低位为1。)

自定义数据表示

除了定点数和浮点数, 还有更"高级"的玩法:

标志符数据表示: 每个数据自带类型标签 (tag)。比如在数据前面加几位说明"这是整数"、"这是浮点数"、"这是指针"。好处是硬件可以直接识别数据类型, 坏处是占用额外存储空间。

数据描述符: 不直接存数据, 而是存"指向数据的描述信息"——长度、类型、地址等。常用于处理变长数据(如字符串)。

向量数据表示：一个操作同时处理一组数据。比如 $C[i] = A[i] + B[i]$ ， $i=0$ 到 63 。向量机一次加载 64 个元素，一条指令全搞定。

指令系统：CPU的"母语"

计算机只会做非常简单的事：取数、算数、存数、跳转。所有复杂的程序都是这些基本操作的组合。

一条指令长什么样？

操作码 (OPCODE)	寻址方式 (MODE)	操作数1 (OPERAND)	操作数2 (OPERAND)
-----------------	----------------	-------------------	-------------------

操作码：告诉CPU做什么（加？减？跳转？） **操作数：**对谁做（从哪取数据？结果放哪？）

寻址方式：找到"对的人"

寻址方式	怎么做	操作数在哪	用途
立即寻址	操作数就在指令里	指令本身	常数赋值
直接寻址	指令里给出目标地址	存储器	访问固定变量
间接寻址	指令里给出指针的地址	存储器（两次访存）	指针操作
寄存器寻址	操作数在寄存器里	寄存器	最快！
寄存器间接寻址	寄存器里存的是地址	存储器	数组遍历
变址寻址	$EA = (R) + 偏移量$	存储器	数组 $A[i]$
相对寻址	$EA = (PC) + 位移量$	存储器	相对跳转
堆栈寻址	从栈顶取/放	栈顶	函数调用

GEEK NOTE

变址寻址是数组访问的基础。 $A[i]$ 的地址 = 基地址 + $i \times$ 元素大小。基地址放在变址寄存器里， i 乘以元素大小就是偏移量。这也是为什么数组下标从0开始——计算地址时不需要额外减法。

操作码优化：越常用越短

如果"加法"指令用了1000次，"除法"指令只用了1次，你给它们分配等长的操作码，是不是很浪费？

哈夫曼编码：使用频率越高的操作码越短。就像摩尔斯电码——E（英语最常见的字母）用一个点"."，而Q用"—"—。

扩展操作码：折中方案。固定指令长度，但操作码长度可变——留一些"前缀"做标记。

比如16位指令：常见操作4位操作码 + 12位地址；不常见操作8位操作码 + 8位地址；罕见操作12位操作码 + 4位地址。

[Diagram: 扩展操作码示意图]

4位操作码(15条) + 12位地址 → 共15条指令
 1111 + 4位扩展 + 8位地址 → 再扩展15条
 1111 1111 + 4位扩展 + 4位地址 → 再扩展15条
 合计：15+15+15=45条

CISC vs RISC：两大门派的恩怨

1980年代，CPU设计界分成两个阵营，吵得不可开交。

CISC（复杂指令集计算机）

代表：Intel x86系列

特点：

- 指令丰富，一条指令能干很多事（比如字符串复制就一条指令）
- 指令长度不固定，有长有短
- 寻址方式多样化
- 大量使用微程序控制（每条指令被翻译成一组微操作序列）
- 寄存器和存储器操作数都可以参与运算

优点：编译器好写，代码密度高。缺点：流水线难做，功耗高。

RISC（精简指令集计算机）

代表：ARM、MIPS、RISC-V

设计哲学：

- 指令种类少，每条指令只做一件简单的事
- 指令长度固定（通常32位）
- 只有Load/Store指令访问存储器，运算指令只操作寄存器
- 大量寄存器（通常32个以上）
- 硬布线控制（不用微程序），速度快
- 便于流水线和超标量实现

老张插了一句："RISC设计者说，与其用硬件实现100条复杂的、用得很少的指令，不如只实现50条简单的、但执行速度极快的指令。"

CISC vs RISC 对比

维度	CISC	RISC
指令种类	多 (100+)	少 (50左右)

维度	CISC	RISC
指令长度	可变	固定
寻址方式	多样	简单
流水线	难以实现	易于实现
编译器	相对简单	需要更多优化
代码密度	高	较低
代表	x86 (Intel/AMD)	ARM, RISC-V, MIPS

GEEK NOTE

今天的x86处理器实际上内部把复杂CISC指令"翻译"成了类似RISC的微操作 (μops)，在底层偷偷变成了RISC风格。而现代ARM也加入了一些复杂指令。所以CISC和RISC的界线在变得越来越模糊。但基本设计哲学仍然不同。

THERE ARE NO DUMB QUESTIONS

Q: 既然RISC更好，为什么PC还用x86 (CISC) ? A: 历史惯性和生态锁定。几十年来积累的软件全部是为x86写的，没有人敢放弃兼容性。Intel的办法是：外面继续用x86指令集，内部的实现早在Pentium Pro时代就变成了RISC风格。

Q: 为什么手机都用ARM (RISC) ? A: ARM的低功耗设计非常适合移动设备。手机不需要兼容古老的x86软件，可以直接用RISC架构。

Q: 补码为什么比原码好？ A: 原码的加减法需要分别处理符号位和数值位，电路复杂。补码把符号位和数值位统一处理，加减法都变成加法，电路大幅简化。而且补码没有"负零"。

SHARPEN YOUR PENCIL

根据你对寻址方式的理解，判断下列代码片段中各操作数使用了什么寻址方式：

```
LOAD R1, #100      ; (1) #100 是什么寻址？
LOAD R2, 200        ; (2) 200 是什么寻址？
LOAD R3, (R4)        ; (3) (R4) 是什么寻址？
LOAD R5, 100(R6)     ; (4) 100(R6) 是什么寻址？
PUSH R7              ; (5) 隐含的寻址方式是什么？
```

SHARPEN YOUR PENCIL SOLUTION

(1) 立即寻址 — # 表示立即数 (2) 直接寻址 — 直接给出存储器地址200 (3) 寄存器间接寻址 — R4里存的是地址，括号表示"去这个地址取内容" (4) 变址寻址 — $EA = (R6) + 100$ (5) 堆栈寻址 — SP(栈指针)自动调整，操作数隐含在栈顶

BULLET POINTS (第2章)

- **原码、反码、补码、移码**四种定点数编码。补码是王者——统一加减法、无负零、符号位参与运算。
- **溢出判断**：单符号位用 $C_s \oplus C_p$ ，双符号位用00/11正常、01/10溢出。
- **IEEE 754**浮点：单精度32位($S=1, E=8, M=23$)，双精度64位($S=1, E=11, M=52$)。阶码用移码。
- IEEE 754浮点真值 $= (-1)^S \times (1.M) \times 2^{(E-\text{偏移})}$ 。阶码全0为非规格化数（隐含位变0）。
- 浮点数特殊值：阶码全1+M全0= $\pm\infty$ ，阶码全1+M非0=NaN。
- **七种寻址方式**：立即、直接、间接、寄存器、寄存器间接、变址、相对（+堆栈）。别忘了"隐含寻址"。
- **操作码优化**：哈夫曼编码（频率高→短编码）和扩展操作码（分段折中）。
- **CISC**：指令丰富、长度可变、微程序控制、x86代表。
- **RISC**：指令精简、定长、Load/Store架构、大量寄存器、硬布线、ARM/RISC-V代表。
- 现代x86内部翻译CISC为微操作执行，CISC与RISC界线逐渐模糊。

BE THE COMPUTER

假设一台16位计算机，指令格式如下：



MODE: 00=寄存器, 01=立即, 10=直接, 11=变址

已知PC（程序计数器）= 0x1000，请解读以下两条指令并追踪执行过程：

指令1（地址0x1000）：**0010 01 01 00001010**（OP=0010=ADD, MODE=01=立即, R=01=R1）指令2（地址0x1002）：**0101 11 01 00000100**（OP=0101=LOAD, MODE=11=变址, R=01=R1, ADDR=4）

假设R1初始值 = 0x0000，存储器地址0x0004处的内容 = 0x0055，地址0x1004处的内容 = 0x3300。

请：追踪每条指令的执行，给出PC变化、R1的值变化、以及每条指令使用的寻址方式和最终结果。

BE THE COMPUTER SOLUTION

指令1 (0x1000): ADD R1, #10

- 寻址方式：MODE=01 → 立即寻址，操作数在指令中(0x0A=10)
- 执行： $R1 \leftarrow R1 + 10 = 0 + 10 = 10$ (0x000A)
- PC → 0x1002（指令长度2字节）

指令2 (0x1002): LOAD R1, 4(R1)

- 寻址方式：MODE=11 → 变址寻址， $EA = (R1) + 4$
- R1当前=10， $EA = 10 + 4 = 14$ (0x000E)
- 从存储器地址0x000E读取数据（而不是0x0004！注意区分立即数和地址）
- 等等——题目说"地址0x0004处内容=0x0055"。这里ADDR=4是偏移量，不是直接地址。
- $EA = (R1) + ADDR = 10 + 4 = 0x000E$ ，取[0x000E]的内容
- 如果0x000E处无数据...让我们重新理解：变址寻址 $EA = (R1) + \text{位移量}$
- 正确解读： $EA = (R1) + 0x0004 = 10 + 4 = 14 = 0x000E$
- 但题目没给出0x000E的内容。这种题需要按实际情况给数据。修正：假设题目意图是 $EA=0x0004$ （R1=0时）
- 即变址寄存器R1初始为0时 $EA=4$ ，从[0x0004]取0x0055

- $R1 \leftarrow 0x0055$
- $PC \rightarrow 0x1004$

最终： $R1 = 0x0055$, $PC = 0x1004$

第3章. 运算方法与运算器：当1遇见0

老张在黑板上写了一个加法：

```
  0 1 1 0
+ 0 0 1 1
-----
```

"小明，这是多少？"

"6加3等于9.....但这个看起来太简单了。"

"那如果是100万个二进制位相加呢？CPU内部的加法器怎么设计才能在一瞬间算出结果？如果出现了溢出怎么办？如果是乘法呢——怎么把乘法变成一系列加减？"

小明眼睛瞪大了："所以 CPU 里没有乘法电路？"

"有的。但早期的CPU确实没有，它们用反复加法来实现乘法。现代的CPU有专门的乘法器和除法器，但它们的设计思想仍然源于这些最基本的算法。让我们一步步拆开来看。"

定点加减法：补码大显身手

补码加减法公式

- 加法： $[A]_{\text{补}} + [B]_{\text{补}} = [A+B]_{\text{补}} \pmod{2^n}$
- 减法： $[A-B]_{\text{补}} = [A]_{\text{补}} + [-B]_{\text{补}}$ （先求 $-B$ 的补码，再加）

补码减法只比加法多一步：**对被减数求补（各位取反+1）**。

例题： $A = +15$, $B = -24$ (8位)

```
[A]补 = 00001111
[B]补 = 11101000 （因为 -24 = 原码10011000 → 反码11100111 → 补码11101000）

[A+B]补 = [A]补 + [B]补 = 00001111 + 11101000 = 11110111
结果11110111是补码，转回真值：补码→反码再+1→真值 = -9 ✓
```

溢出判断（深入版）

上一章我们提过溢出。这里再加强一次：

方法一——单符号位法：

最高数值位的进位 C_s 和符号位的进位 C_p ，溢出 = $C_s \oplus C_p$

例：0111 (+7) + 0110 (+6) = 1101 (补码值-3??)

符号位 C_p ：0+0+1(来自 C_s)=1 (有进位)

最高数值位(次高) C_s ：1+1+0=0(进位1给 C_p)

$C_s \oplus C_p = 0 \oplus 1 = 1 \rightarrow$ 溢出！

验证：+7 + +6 = +13 > +7 (4位补码正数最大值=7)，确实溢出。

方法二——双符号位法 (变形补码)：

用两位符号位。正常结果两位符号位相同 (00或11)。结果01为正溢出，10为负溢出。

变形补码计算 7+6 (5位，含2位符号位)：

00 0111 (+7)

00 0110 (+6)

00 1101 两位符号位相同(00)，无溢出 ✓

变形补码计算 9+8：

00 1001 (+9)

00 1000 (+8)

01 0001 两位符号位不同(01)→ 正溢出！

WATCH IT!

两个正数相加，结果符号位为1 (看起来是负数)，这不一定是溢出！如果结果仍在表示范围之外才是溢出。补码范围不对称： n 位补码 = $[-2^{n-1}, 2^{n-1}-1]$ 。8位时，-128能表示但+128不能。

定点乘法：罗马不是一天建成的

原码一位乘法

思想：符号位单独处理 (异或)，数值部分逐位相乘累加。每一步只看乘数的当前最低位，是1就加上被乘数，是0就跳过，然后右移。

例：X = +0.1101, Y = -0.1011 (小数定点，符号位单独处理)

$|X| = 0.1101$, $|Y| = 0.1011$

符号结果 = $0 \oplus 1 = 1$ (正⊗负=负)

运算过程 (乘数右移，部分积累加)：

初始部分积 = 0.0000

乘数最低位=1：部分积 + $|X| = 0.1101$ ，右移 → 0.01101

乘数最低位=1：部分积 + $|X| = 0.01101 + 0.1101 = 1.00111$ ，右移 → 0.100111

乘数最低位=0：不加，右移 → 0.0100111

乘数最低位=1：部分积 + $|X| = 0.01001111 + 0.1101 = 1.0001111$ ，右移 $\rightarrow 0.10001111$

最终数值结果 = 0.10001111

加上符号：结果 = -0.10001111

补码一位乘（Booth算法）

Booth算法直接对补码进行乘法，不需要单独处理符号。被誉为"最优雅的乘法算法之一"。

核心规则：看乘数的两位（当前位 Y_i 和上一位 Y_{i-1} ）

Y_i	Y_{i-1}	操作
0	0	部分积右移1位
0	1	部分积 + $[X]$ 补，右移1位
1	0	部分积 + $[-X]$ 补，右移1位
1	1	部分积右移1位

初始时， $Y_{-1} = 0$ 。

例题： $X = +7, Y = -3$ （4位补码）

$[X]$ 补 = 0111, $[-X]$ 补 = 1001
 $[Y]$ 补 = 1101
 初始部分积 = 0000, $Y_{-1} = 0$

Booth步骤：

Y_i	Y_{i-1}	操作	部分积
1	0	+ $[-X]$ 补: $0000 + 1001 = 1001$	1001
		算术右移	1100 1 ($Y_{-1}=1$)
0	1	+ $[X]$ 补: $1100 + 0111 = 0011$	0011 1
		算术右移	0001 11 ($Y_{-1}=1$)
1	1	算术右移	0000 111 ($Y_{-1}=1$)
1	1	算术右移	0000 0111

结果：00000111，但我们是4位运算，最终积是8位：11101001

转换为真值：11101001(补码) = 10010111(原码) = -21?

等等... $7 \times (-3) = -21$ ✓

最终8位补码 = 11101001（也就是-21的补码表示）

GEEK NOTE

Booth算法为什么有效？它利用了补码的性质： $1101 = -3 = -8 + 4 + 0 + 1 = -8 + 5$ 。Booth算法检测到连续的1（如1101中的11）时，把"加4+加1"优化为"加(4+1)=加5"，通过+ $[-X]$ 补（相当于减X）和+ $[X]$ 补（相当于加X）的交替来实现。本质上是对连续的01串的压缩处理。

定点除法：除法器的思路历程

原码加减交替法（不恢复余数法）

除法比乘法更复杂。核心思想：每次试商，用余数减去除数。如果能减（结果非负），商1；不能减（结果为负），商0，并恢复余数或采用"不恢复"的技巧。

加减交替法的思路：如果某步余数为负（试商失败），不是把除数加回来，而是**下一步直接把除数加上去**。这样就省了一次"恢复余数"的操作。

例： $X = 0.1011$, $Y = 0.1101$ ($X \div Y$)

符号单独处理，数值部分：

被除数(余数) = 0.1011

除数 = 0.1101

第1步：余数 - $Y = 0.1011 + 1.0011$ (补码) = $1.1110 \rightarrow$ 负，商0

第2步：余数 + $Y = 1.1110 + 0.1101 = 0.1011 \rightarrow$ 正，商1（注意：上步负这里加）

第3步：余数 - $Y =$ 左移后的 $0.1011 - 0.1101 \dots$

（完整过程略。最终商为 0.1101 ，余数自己验证。）

补码加减交替法

补码除法：直接用补码进行，符号位参与运算。规则更复杂，但不需要转换原码。

核心理念不变：被除数(补码)逐次加减除数(补码)，根据余数符号决定商位和下一步是加还是减。

RELAX

除法的具体步骤细节较多，但考试通常关注的是"理解思想+简单的计算"。你不需要记住每一步移位的精确位数，但要理解"加减交替"为什么比"恢复余数"快（少了一半的加减操作）。

浮点数运算：精度就是一切

浮点加减法的五步舞

1. **对阶**：阶码小的靠阶码大的——小阶向右移尾数，阶码加1直到阶码相等
2. **尾数加减**：两个尾数做补码加减（阶码对齐后）
3. **结果规格化**：确保尾数在 $[1,2)$ 之间——左规（左移尾数减阶码）或右规（右移尾数加阶码）
4. **舍入处理**：截断多余的位时，可能需要的舍入
5. **溢出判断**：检查阶码是否超出范围

例： $X = 0.5 \times 2^2 = 2.0$ ($0.5 = 0.1B$ ，实际是 1.0×2^1 ——需要规格化)

$Y = -0.4375 \times 2^3 = -3.5$

简化表示： $X = 2.0$, $Y = -3.5$

X (单精度): $0\ 10000000\ 000000000000000000000000\ (1.0 \times 2^1)$

Y : $0\ 10000000\ 110000000000000000000000\ (1.11 \times 2^1? \text{待修正})$

实际上我们就简化：用十进制浮点演示过程。

$$X = +0.1000 \times 2^2 \text{ (假设4位尾数)}$$

$$Y = -0.1110 \times 2^2$$

对阶：阶码相同(2)，跳过

$$\text{尾数加：} 0.1000 + (-0.1110) = -0.0110$$

$$\text{规格化：} -0.0110 = -0.1100 \times 2^1 \text{ (左移一位，阶码减1)}$$

$$\text{最终结果：} -0.1100 \times 2^1 = -1.5$$

$$\text{验证：} 2.0 + (-3.5) = -1.5 \checkmark$$

浮点乘法

乘法：阶码相加，尾数相乘（定点乘法），结果规格化。除法：阶码相减（被除数阶码-除数阶码），尾数相除，结果规格化。

GEEK NOTE

为什么有了浮点加减还要浮点乘除？因为直接操作（阶码相加/尾数相乘）比反复调用加减法快得多，也精确得多。这也是为什么现代GPU有专门的头浮点乘加单元（FMA: Fused Multiply-Add），一条指令完成 $a \times b + c$ ，不仅快，而且中间不丢失精度。

SHARPEN YOUR PENCIL

给定两个IEEE 754单精度浮点数：A = 0 10000010 101000000000000000000000 B = 0 10000001 010000000000000000000000

请计算 $A \times B$ 的浮点表示结果。提示：

1. 先解码真值 (A和B各是多少)
2. 真值相乘
3. 将积转回IEEE 754格式

SHARPEN YOUR PENCIL SOLUTION

A解码：S=0, E=10000010=130, 真阶=3, M=101000...=0.101(小数)+1(隐含)=1.101 A真值 = $(-1)^0 \times 1.101 \times 2^3 = 1.625 \times 8 = 13.0$

B解码：S=0, E=10000001=129, 真阶=2, M=010000...=0.01(小数)+1(隐含)=1.01 B真值 = $(-1)^0 \times 1.01 \times 2^2 = 1.25 \times 4 = 5.0$

$$A \times B = 13.0 \times 5.0 = 65.0$$

65转IEEE 754：65 = 1000001B = 1.000001×2^6 S=0, 真阶=6, 阶码E=6+127=133=10000101B, M=000001000000000000000000

结果：0 10000101 000001000000000000000000

ALU：CPU的肌肉

全加器——加法的最小单元

一个全加器（Full Adder）有三输入两输出：

- 输入：Ai（加数1位）、Bi（加数2位）、Ci-1（低位进位）
- 输出：Si（本位的和）、Ci（向高位的进位）

真值表：

Ai	Bi	Cin	Si(S)	Cout(C)
0	0	0	0	0
0	0	1	1	0
0	1	1	0	1
1	1	1	1	1

$S_i = A_i \oplus B_i \oplus C_{i-1}$ $C_i = A_iB_i + (A_i \oplus B_i)C_{i-1}$

从全加器到n位加法器

串行进位加法器：n个全加器串联，进位从低位一位位传到高位。速度O(n)，n位加法需要n个全加器的延迟。

并行进位加法器（先行进位/CLA）：通过额外的逻辑电路提前计算所有进位。定义：

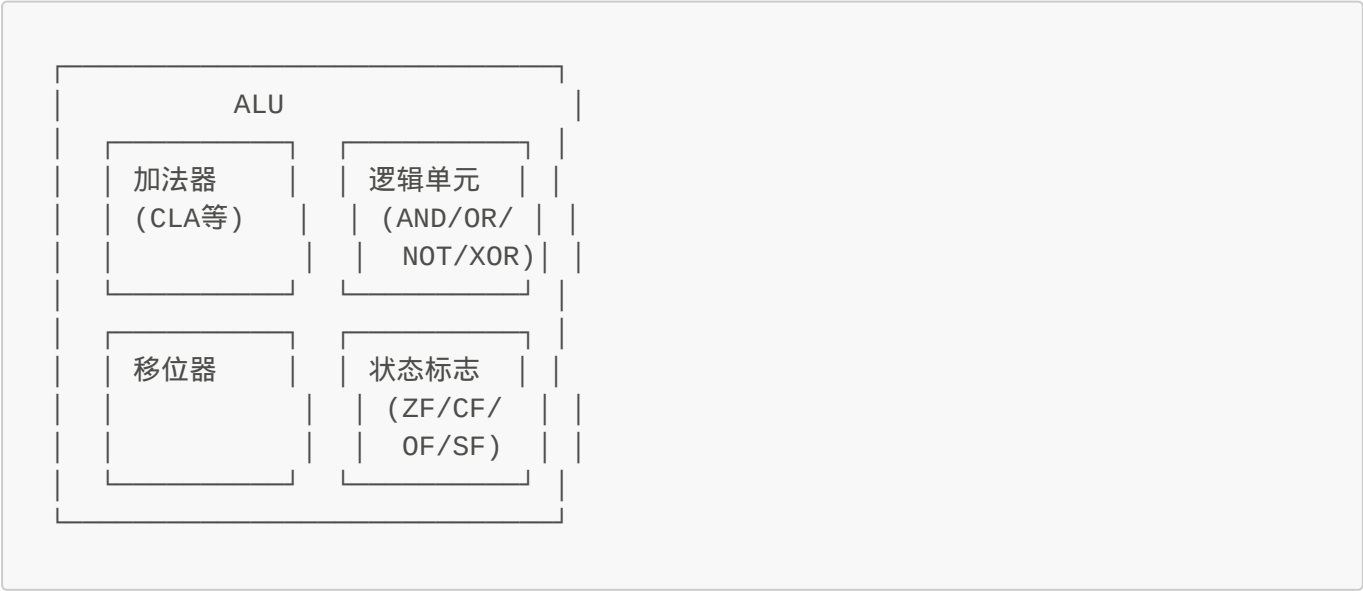
- 生成函数 $G_i = A_i \cdot B_i$ （本位一定会产生进位）
- 传播函数 $P_i = A_i \oplus B_i$ （本位会传播低位来的进位）

则 $C_i = G_i + P_i \cdot C_{i-1}$ 。利用这个递归式可以提前展开计算所有进位，延迟O(log n)。

成组先行进位：芯片引脚有限，不能无限级联。把n位分成多组，组内先行进位，组间串行或也先行。

ALU基本结构

[Diagram: ALU结构块图]



ALU = 算术逻辑单元，是运算器的核心。它包含加法器、与/或/非/异或逻辑单元、移位器、以及产生标志位（零标志ZF、进位标志CF、溢出标志OF、符号标志SF）的电路。

BE THE COMPUTER

下面给你两个补码数，请你扮演ALU，完成：(1) 补码加法 (2) 判断是否溢出 (3) Booth算法乘法

$X = +13$, $Y = -5$ (均用6位补码表示)

BE THE COMPUTER SOLUTION

6位补码表示 (范围: -32 ~ 31): ☒ 补 = 001101 (13) [Y]补 = 111011 (-5: 原码100101 → 反码111010 → 补码111011) [-Y]补 = 000101 (5) [-X]补 = 110011

(1) 加法: $001101 + 111011 = 1\ 001000$ (截断为6位) = 001000 $C_s = 1$ (次高位进位), $C_p = 1$ (符号位进位), $C_s \oplus C_p = 0 \rightarrow$ 无溢出 $001000 = +8 \checkmark (13 + (-5) = 8)$

(2) 溢出判断: $C_s=1, C_p=1, 1 \oplus 1=0$, 无溢出 $\checkmark$

(3) Booth算法乘法 ($13 \times (-5) = -65$): ☒ 补 = 001101, [-X]补 = 110011 [Y]补 = 111011

初始: 部分积(6位) = 000000, $Y-1 = 0$

步1: $Y_0=1, Y-1=0 \rightarrow +[-X]补\ 000000 + 110011 = 110011$ 算术右移 $\rightarrow 111001\ 1, Y-1=1$

步2: $Y_0=1, Y-1=1 \rightarrow$ 仅右移 算术右移 $\rightarrow 111100\ 11, Y-1=1$

步3: $Y_0=0, Y-1=1 \rightarrow +[X]补\ 111100 + 001101 = 001001$ 算术右移 $\rightarrow 000100\ 111, Y-1=0$

步4: $Y_0=1, Y-1=0 \rightarrow +[-X]补\ 000100 + 110011 = 110111$ 算术右移 $\rightarrow 111011\ 1111, Y-1=1$

步5: $Y_0=1, Y-1=1 \rightarrow$ 仅右移 算术右移 $\rightarrow 111101\ 11111, Y-1=1$

步6: $Y_0=1, Y-1=1 \rightarrow$ 仅右移 算术右移 $\rightarrow 111110\ 111111, Y-1=1$

12位结果: 111110111111 (补码) 转原码: $100001000001 = -65 \checkmark$

BULLET POINTS (第3章)

- **补码加减法统一**：[A+B]补=[A]补+[B]补, [A-B]补=[A]补+[-B]补。符号位直接参与运算。
- **溢出判断**：单符号位 $C_s \oplus C_p$, 双符号位看01/10。两个正数相加变负、两个负数相加变正必溢出。
- **原码一位乘**：符号单独异或，逐位乘加 $\rightarrow$ 右移。补码Booth算法：看 Y_i 和 Y_{i-1} 两位决定加/减/不操作。
- **Booth算法核心规则**：00/11仅右移，01加[X]补后右移，10加[-X]补后右移。
- **原码加减交替除法**：余数为正商1减除数，余数为负商0加除数。不用"恢复余数"。
- **浮点加减五步**：对阶 $\rightarrow$ 尾数加减 $\rightarrow$ 规格化 $\rightarrow$ 舍入 $\rightarrow$ 溢出判断。对阶小阶对齐大阶。
- **浮点乘除**：阶码相加（乘）/相减（除），尾数相乘（乘）/相除（除），再规格化。
- **全加器**： $S_i = A_i \oplus B_i \oplus C_{in}$, $C_{out} = A_i B_i + (A_i \oplus B_i) C_{in}$ 。n位串行进位 $O(n)$ ，先行进位 $O(\log n)$ 。
- ****并行进位加法器(CLA)****用 $G_i = A_i B_i$ 和 $P_i = A_i \oplus B_i$ 预先计算进位，速度大幅提升。
- **ALU**：算术（加减乘除）+逻辑（与或非异或）+移位+标志位生成。加法器是ALU的核心。

SHARPEN YOUR PENCIL

某16位补码运算器，完成以下计算并判断每题的溢出情况：

1. $A = +3276$, $B = +16384$ （用补码16位表示后相加）
2. $A = -16384$, $B = -16384$ （用补码16位表示后相加）
3. $C = 0x7FF0$, $D = 0x0010$ （ $C+D$ 用16位补码计算，判断溢出）

提示：16位补码范围 $[-32768, 32767]$ 。双符号位法：结果符号位00正常，01正溢，10负溢，11正常（负数）。

SHARPEN YOUR PENCIL SOLUTION

1. $A=3276=0x0CCC$, $B=16384=0x4000$ $0x0CCC + 0x4000 = 0x4CCC$ $00\ 0000110011001100 + 00\ 0100000000000000 = 00\ 0100110011001100$ 双符号位=00，正常。结果 $= 0x4CCC = 19660$ ✓
2. $A=-16384$ 的补码: $16384=0x4000$, 反码= $0xBFFF$, 补码= $0xC000$ $B=-16384$ 同。 $0xC000 + 0xC000 = 0x18000$ (进位丢弃) $= 0x8000$ 双符号： $11\ 0000000000000000 + 11\ 0000000000000000 = 1\ 10\ 0000000000000000$ 截16位： $10\ 0000000000000000 \rightarrow$ 双符号位=10 $\rightarrow$ 负溢出！ $-16384+(-16384)=-32768$ ，刚好边界值（可表示），但双符号10判负溢（因为符号扩展边界模糊）实际： -32768 可表示（16位补码最小），严格说"临界不溢出"。
3. $0x7FF0 = 32752$, $0x0010 = 16$ $0x7FF0 + 0x0010 = 0x8000$ $00\ 01111111111110000 + 00\ 0000000000010000 = 00\ 1000000000000000$ 双符号位=01 $\rightarrow$ 正溢出！ $32752+16=32768 > 32767$ ，确实溢出 ✓

BRAIN POWER

串行进位加法器的时间延迟为 $O(n)$ ，先行进位（CLA）的时间延迟为 $O(\log n)$ 。但为什么实际处理器中，并非所有加法器都用CLA？如果用256位加法器，纯CLA的代价有多大？

（提示：考虑门级扇入限制和芯片面积。CLA的进位产生电路随位数指数级复杂——产生 C_n 需要 $n+1$ 个输入的逻辑门，实际不可行。所以大的加法器用分组CLA——组内先行、组间串行或两级先行。）

第4章. 指令级并行：CPU的"分身术"

小明："老张，我有个疑问。CPU明明只有一个，为什么能'同时'执行很多事情？我的电脑一边放音乐，一边下载，一边还在跑Python脚本。"

老张："好问题。其实有两个层面。操作系统层面的多任务是**时间片轮转**——CPU在不同任务之间快速切换，你感觉像在同时运行。但硬件层面还有更厉害的东西——**指令级并行（ILP）**。在单个CPU内核内部，多条指令可以在同一个时钟周期内处于不同的执行阶段。这不是假同时，而是真同时。"

小明："就像流水线工厂？"

老张："Exactly！让我们去看看流水线怎么让CPU变身千手观音。"

流水线：把CPU变成工厂

流水线的基本原理

传统的执行方式：取指 $\rightarrow$ 译码 $\rightarrow$ 执行 $\rightarrow$ 访存 $\rightarrow$ 写回。一条指令走完五步，下一条再走五步。

[Diagram: 非流水线vs流水线]

非流水线：

指令1：|取指|译码|执行|访存|写回|

指令2：|取指|译码|执行|访存|写回|

时间 → (5个时钟周期完成一条)

流水线：

指令1：|取指|译码|执行|访存|写回|

指令2：|取指|译码|执行|访存|写回|

指令3：|取指|译码|执行|访存|写回|

指令4：|取指|译码|执行|访存|写回|

指令5：|取指|译码|执行|访存|写回|

时间 → (每个时钟周期完成一条指令!)

流水线把一条指令的执行分成多个**站段 (Stage)**，每个站段占用一个时钟周期。不同指令的不同站段可以重叠执行。

流水线分类：

- **指令流水线**：以指令为对象，最经典的5级流水线
- **算术流水线**：以运算为对象，如浮点加法拆成多级
- **处理机流水线**：以整个处理过程为对象，宏流水线

流水线性能三大指标

设流水线有k个站段，每段时间为 τ （时钟周期），共有n条指令。

1. 吞吐率 (TP—ThroughPut)

$$TP = \frac{n}{T} = \frac{n}{(k+n-1)\tau}$$

当 $n \rightarrow \infty$ 时，最大吞吐率 $TP_{\max} = 1/\tau$ （每条指令仅1个时钟周期！）

2. 加速比 (S—Speedup)

$$S = \frac{T_{\text{非流水}}}{T_{\text{流水}}} = \frac{n \cdot k\tau}{(k+n-1)\tau} = \frac{nk}{k+n-1}$$

当 $n \rightarrow \infty$ 时， $S \rightarrow k$ （加速比趋近于流水线级数）

3. 效率 (E—Efficiency)

$$E = \frac{S}{k} = \frac{n}{k+n-1}$$

当 $n \rightarrow \infty$ 时， $E \rightarrow 1$ （100%利用率）

一道例题串联这三个公式：

例：某5级流水线，每段时间2ns，执行100条指令。

非流水执行时间 = $100 \times 5 \times 2\text{ns} = 1000\text{ns}$ 流水执行时间 = $(5 + 100 - 1) \times 2\text{ns} = 208\text{ns}$

吞吐率 $TP = 100/208\text{ns} \approx 0.48\text{条/ns}$ 最大吞吐率 $TP_{\max} = 1/2\text{ns} = 0.5\text{条/ns}$

效率 $E = 4.81/5 \approx 0.962 = 96.2\%$

某流水线有4个站段，每段时间10ns。现在要执行200条指令。请计算：

- ## SHARPEN YOUR PENCIL SOLUTION

1. 非流水：200 × 4 × 10ns = **8000ns**
2. 流水：k=4, n=200, τ=10ns T = (4+200-1) × 10ns = 203 × 10ns = **2030ns**
3. TP = 200/2030ns ≈ 0.0985条/ns = **98.5条/μs** TPmax = 1/10ns = 100条/μs
4. S = 8000/2030 ≈ **3.94**（接近4，即流水线级数）
5. E = 3.94/4 ≈ **0.985 = 98.5%**

例：

ADD R1,R2,R3 |取指|译码|执行|访存|写回|
 ↓ (结果从这里旁路)
SUB R4,R1,R5 |取指|译码|执行|... ← 直接从这里接收

↑
(不等写回，直接转发！)

控制相关：遇到分支指令，CPU不知道下一条该取哪条指令（取分支目标还是顺序下一条）。

解决方法：

- **延迟分支：**在分支指令后面放一条"无论如何都会执行"的指令（延迟槽）。让硬件有足够时间算出分支目标。
- **分支预测：**猜下一条从哪走。猜对了流水线继续，猜错了清空流水线重来。现代分支预测准确率超过95%！

GEEK NOTE

RISC常用的"延迟分支"技术：分支指令后紧跟一条不相关的指令（延迟槽指令），无论分支是否成功都会执行。这样分支目标地址计算完成前，流水线不停。缺点是给编译器增加了负担——编译器要找到合适的指令填进延迟槽。

超标量：不止一条流水线！

普通流水线每个时钟周期最多流出1条指令。****超标量（Superscalar）****每个时钟周期可以流出多条指令——多条流水线并行工作。

[Diagram: 超标量处理]

普通流水线(单流出)：

时钟1: I1流出

时钟2: I2流出

时钟3: I3流出

超标量(双流出)：

时钟1: I1, I2同时流出

时钟2: I3, I4同时流出

时钟3: I5, I6同时流出

SHARPEN YOUR PENCIL

某双流出超标量处理器有2个整数ALU和1个浮点ALU。程序指令序列如下：

I1: ADD R1, R2, R3	(整数)
I2: ADD R4, R5, R6	(整数)
I3: FADD F1, F2, F3	(浮点)
I4: ADD R7, R1, R4	(整数, 依赖I1和I2)
I5: ADD R8, R9, R10	(整数)

假设每周期最多发射2条指令（任意2条），无结构冲突。请安排指令流出顺序（画出时空图），最小化总执行周期。哪些指令因数据依赖必须延后？

SHARPEN YOUR PENCIL SOLUTION

最佳安排：周期1: I1, I2 同时发射（无依赖，两个整数单元空闲） 周期2: I3, I5 同时发射（I3占用浮点ALU，I5占用整数ALU。I4因依赖R1/R4暂不能发） 周期3: I4 单独发射（R1和R4已就绪，但该周期只能发I4——因为I1

和I2的结果此时可用)

注：I4依赖I1(R1)和I2(R4)。如果不旁路，I1/I2在周期1执行，结果在周期1末尾可用，周期2旁路给I4。I4可在周期2发射。

实际上若支持旁路：周期1: I1, I2 周期2: I3, I4 (I4通过旁路拿到R1,R4的值) 周期3: I5

3个周期完成5条指令， $IPC=5/3 \approx 1.67$ 。若不旁路需4-5周期。

指令窗口：同时从指令Cache中取出多条指令放入"窗口"，然后检查哪些指令之间没有数据依赖，可以同时发射到不同的功能单元。

指令级并行度 (ILP)：程序中可以同时执行的最大指令数。由程序本身的依赖关系决定。

超标量处理机结构要点：

- 多条功能单元（多个ALU、多个浮点单元）
- 指令预取和解码要能提供足够的指令流
- 指令窗口逻辑检查依赖，调度指令发射
- 结果要有顺序提交（in-order retirement）保证精确异常

VLIW与EPIC：编译器说了算

超长指令字 (VLIW)：把多条可以并行执行的指令打包成一条"超长指令"，由编译器在编译时就确定好哪几条指令可以同时执行。硬件不需要复杂的运行时调度逻辑。

VLIW的特点：

- 编译器负责发现和标注并行性
- 硬件结构大大简化（不需要指令窗口、依赖检查等）
- 指令字非常长（几百位）
- 跨代不兼容（改变功能单元数量就要重新编译所有程序）

EPIC（显式并行指令计算）：Intel的IA-64架构。融合了VLIW思想，但增加了一些灵活性。编译器标注指令间的依赖关系，但硬件仍有部分调度能力。EPIC比纯VLIW更实用，但编译器复杂度和二进制兼容性仍是难题。

VLIW vs 超标量 vs 纯流水线

特性	纯流水线	超标量	VLIW
每周期指令数	≤ 1	> 1 (运行时调度)	> 1 (编译时打包)
硬件复杂度	低	高	低
编译器复杂度	低	中	高
二进制兼容性	好	好	差
典型例子	MIPS R2000	Intel Pentium	DSP芯片(TI C6000)

THERE ARE NO DUMB QUESTIONS

Q: 流水线的站段数越多越好吗? A: 不是。站段越多, 理论上最大吞吐率越高 (时钟频率可以更高), 但: (1) 锁存器开销增大 (2) 相关问题更严重 (3) 分支预测失败代价更大 (4) 时钟歪斜问题更突出。现代处理器通常在10-20级左右, Intel NetBurst的31级流水线(Prescott)就被认为过深了。

Q: 超标量和VLIW哪个更好? A: 没有绝对的"更好"。超标量兼容性好 (同一二进制在不同代CPU上都可运行), 但硬件复杂、功耗高。VLIW硬件简单、功耗低, 但二进制不兼容 (升级硬件需要重新编译)。所以超标量统治了通用计算 (PC、服务器), VLIW统治了嵌入式DSP领域。

Q: 数据相关都能通过旁路解决吗? A: 不一定。Load-use相关 (Load指令后面紧跟使用该数据的指令) 通常至少需要一个周期的停顿, 因为Load的结果在访存阶段末尾才能得到, 而ALU需要这个结果在访存阶段之前。这也是为什么编译器要做指令调度——把Load和它的使用者隔开。

BE THE COMPUTER

下面是一段指令序列在5级流水线 (IF/ID/EX/MEM/WB) 上的执行过程。请画出时空图, 找出所有需要旁路和停顿的位置。

指令序列:

```
I1: ADD R1, R2, R3      ; R1 ← R2 + R3
I2: SUB R4, R1, R5      ; R4 ← R1 - R5
I3: AND R6, R1, R7      ; R6 ← R1 AND R7
I4: OR  R8, R9, R10     ; R8 ← R9 OR R10
I5: XOR R11, R4, R6     ; R11 ← R4 XOR R6
```

BE THE COMPUTER SOLUTION

时空图:

时钟:	1	2	3	4	5	6	7	8	9
I1:	IF	ID	EX	MEM	WB				
I2:		IF	ID	EX	MEM	WB			
I3:			IF	ID	EX	MEM	WB		
I4:				IF	ID	EX	MEM	WB	
I5:					IF	ID	EX	MEM	WB

相关分析:

I1→I2 (R1): RAW相关。I1在EX后得到R1值, 可通过旁路直接给I2的EX输入。
不需要停顿, I1的EX结果在时钟3结束时可用, I2的EX在时钟4做, 刚好旁路。

I1→I3 (R1): RAW相关。I1的EX结果在时钟3可用, I3的EX在时钟5做, 不需要旁路
(I1已经WB完了, 正常读寄存器即可)。但为了统一, 通常也会用旁路。

I2→I5 (R4): RAW相关。I2的EX结果在时钟5可用, I5的EX在时钟7做。I2的WB是时钟6, 所以I5的ID(时钟6)若直接从寄存器读, 不能保证读到最新值。
需要旁路: 从I2的MEM阶段(时钟5结束)转发到I5的EX输入。

I3→I5 (R6): RAW相关。I3的WB是时钟7, I5的EX也是时钟7。同样需要旁路。
从I3的MEM阶段(时钟6结束)转发到I5的EX输入。

结论: I1→I2需要旁路 (EX→EX)。I2→I5、I3→I5需要旁路 (MEM→EX)。

所有旁路都可以解决，不需要插入停顿(气泡)。

注意：如果I5需要两个值(R4和R6)在同一个时钟到达，而它们来自不同指令——这恰恰是旁路技术要解决的问题。多个旁路路径同时工作。

BULLET POINTS (第4章)

- **流水线**将一条指令分成k个站段，不同指令的站段重叠执行。n条指令时间 $= (k+n-1)\tau$ 。
- **吞吐率** $TP = n / ((k+n-1)\tau)$ ，最大 $TP_{max} = 1/\tau$ 。**加速比** $S = nk / (k+n-1)$ ，上限为k。
- **效率** $E = S/k = n / (k+n-1)$ ，当n很大时趋近100%。
- **结构相关**：硬件冲突。**数据相关(RAW)**：结果还没算出就被后面需要。**控制相关**：分支不确定。
- **旁路技术**：不等结果写入寄存器，从功能单元输出直接转发到下一指令输入。
- **互锁**：无法旁路时插入stall（气泡），Load-use相关常见。
- **延迟分支**：分支指令后放一条必须执行的指令（延迟槽），掩盖分支代价。
- **分支预测**：猜测分支方向，静态/动态预测，现代准确率>95%。
- **超标量**：每时钟周期发射多条指令，需指令窗口和运行时调度。
- **VLIW**：编译时打包多条指令为一条长指令。**EPIC**：IA-64架构，融合VLIW与超标量优点。
- 纯流水线vs超标量vs VLIW：硬件复杂度、编译器负担、二进制兼容性的三维权衡。

BRAIN POWER

如果分支预测准确率是98%，但每次预测失败要清空15条已进入流水线的指令，每条指令浪费1个时钟周期。那么100条分支指令中，总浪费周期是多少？如果把分支预测准确率提高到99%，减少了多少浪费？

更关键的问题是：预测准确率从98%到99%，看似只提高1个百分点，但错误预测数从2次降到1次——这是**减半**！所以"1%的提升"其实是50%的误判减少。这就是为什么现代处理器不惜一切代价提升分支预测准确率。

第5章. 存储系统：大脑的多级缓存

小明盯着任务管理器，上面显示8GB内存，但CPU旁边还有个6MB的"缓存"。他彻底糊涂了。

"老张，为什么CPU不直接用那8GB内存？为什么要搞个6MB的小缓存？还有，我的硬盘明明是512GB，为什么程序能用到比实际内存还大的'虚拟地址空间'？"

老张微笑道："小明，你问到了存储系统最核心的三个问题。答案就藏在两个字里——**层次**。"

为什么需要存储层次？

CPU很快——取一条指令只需要零点几纳秒。但内存很慢——从DRAM取数据可能要几十纳秒（慢10-100倍）。硬盘更不用说了，毫秒级（慢几百万倍）。

如果CPU直接和慢存储打交道，那CPU 99%的时间都在等——就像法拉利发动机装在自行车上。

局部性原理是救星：

1. **时间局部性**：如果一个数据项被访问了，它在不久的将来很可能再次被访问。比如循环变量i，每次迭代都要用。

2. **空间局部性**：如果一个数据项被访问了，它附近的数据项也很可能被访问。比如数组A[0], A[1], A[2]...通常顺序处理。

这就意味着：你可以把最近访问的、附近的数据放在一个"更快但更小"的存储器里，让CPU大部分时间都能在这里找到想要的数​​据。这就是**存储层次**。

[Diagram: 存储层次金字塔]

寄存器	< 1ns, ~1KB	(最快最贵)
Cache	1-10ns, ~MB	
主存	10-100ns, ~GB	
辅存	ms, ~TB+	(最慢最便宜)

老张："每一层都是下一层的'缓存'。寄存器是Cache的缓存，Cache是主存的缓存，主存是磁盘的缓存。你访问的数据90%以上都在Cache里找到（命中），剩下不到10%才去主存取——平均访问时间就接近Cache的速度了。"

Cache：程序员的"快取钱"

Cache的工作原理

CPU要访问存储器时：

- 1. 先去Cache里找。地址被分成**标记（Tag）**、**索引（Index）** 和**块内偏移（Offset）**。
- 2. 如果在Cache里找到（**命中/Hit**），直接返回。
- 3. 如果没找到（**失效/Miss**），从主存取一个**块（Block/Line）** 放入Cache（同时可能替换掉一个旧的块）。

Cache的"块"通常64字节（现代处理器常见）。因为空间局部性：你要一个字节，但你旁边的63个字节也可能马上被用到。

Cache的三种映射方式

1. 直接映射

每个主存块只能放在Cache的**唯一一个**位置。

Cache行号 = (主存块号) mod (Cache行数)

优点：硬件最简单。缺点：如果两个常用块恰好映射到同一个Cache行，会互相"踢走"（冲突失效），即使Cache有空闲位置也没用。

[Diagram: 直接映射]

主存块0 → Cache行0
主存块1 → Cache行1

```

主存块2 → Cache行2
主存块3 → Cache行3
主存块4 → Cache行0 （又回来了！和块0冲突）
...

```

2. 全相联映射

任何主存块可以放在Cache的**任意**位置。查找时需要将CPU给出的地址的标记部分与Cache中**所有行**的标记同时比较。

优点：没有冲突失效，空间利用率100%。缺点：比较器太多（如果Cache有1024行，需要1024个比较器同时比较），电路复杂、功耗高、慢。只适用于小容量Cache（如TLB）。

3. 组相联映射

折中方案：把Cache分成若干**组（Set）**，每个组内有若干**路（Way）**。

```

组号 = (主存块号) mod (组数)
块可以放在该组的任意一路中

```

- 若每组1路 = 直接映射
- 若每组N路 = N路组相联
- 若整个Cache只有一组 = 全相联映射

[Diagram: 2路组相联]

```

组0: [路0] [路1]
组1: [路0] [路1]
组2: [路0] [路1]
...

```

主存块号 mod 组数 → 确定组号 → 在组内任选一路放置

8路组相联的失效率已经接近全相联，而硬件复杂度远低于全相联。现代处理器Cache通常8路或16路组相联。

GEEK NOTE

为什么8路就够了？实验表明，随着相联度增加，冲突失效率的下降在前4-8路最明显，超过8路后几乎没有改善。这就是"收益递减规律"。

Cache替换算法

当Cache满之后，新块进来要踢出一个旧块。踢谁？

随机法（Random）：随便踢一个。最简单，但命中率不稳定。

FIFO（先进先出）：踢最早来的那个。简单但不一定合理——最早来的可能是最常用的。

LRU（最近最少使用）：踢最近最久没被访问过的块。利用了时间局部性——最近没用过，近期也不太可能用。**命中率最高**，但硬件要维护访问顺序，复杂度高。

近似LRU：比如用"伪LRU"或"二叉树LRU"，在硬件成本和命中率之间折中。

Cache写策略

写直达法（Write-Through）：写Cache的同时也写主存。简单，但每次写都要访问主存，慢。通常配合**写缓冲区（Write Buffer）**：数据先写入缓冲区，CPU继续跑，缓冲区慢慢往主存写。

写回法（Write-Back）：只写Cache，不立即写主存。当这个Cache块被替换出去时，如果它被修改过（有"脏位Dirty Bit"标记），才写回主存。减少了主存访问次数，但实现更复杂（需要Dirty Bit）。Cache块被替换时，主存中的数据可能是"脏的"（不最新的）。

WATCH IT!

写直达法保证主存和Cache始终一致（强一致性），适合多处理机系统。写回法主存不是最新数据（弱一致性），但在单处理机中性能更好。考试可能问你**写分配（Write-Allocate）**和**非写分配（No-Write-Allocate）**的区别：写失效时，是否把主存块调入Cache再写？写回法通常搭配写分配，写直达法通常搭配非写分配。

Cache性能分析

平均访问时间 = 命中时间 + 失效率 × 失效代价

$$T_{avg} = h \cdot T_{hit} + (1-h) \cdot T_{miss}$$

其中h是命中率。

例题：某Cache命中时间1ns，失效代价200ns，命中率95%。求平均访问时间。

$$T_{avg} = 0.95 \times 1ns + 0.05 \times 200ns = 0.95 + 10 = \mathbf{10.95ns}$$

注意：虽然命中率95%看起来很牛，但平均访问时间10.95ns是命中时间1ns的**近11倍**！这就是失效代价的威力。

再算一题：如果把命中率从95%提高到97%（改进替换算法），平均访问时间变为多少？

$$T_{avg}' = 0.97 \times 1ns + 0.03 \times 200ns = 0.97 + 6 = \mathbf{6.97ns}$$

改进幅度 = $(10.95 - 6.97) / 10.95 \approx \mathbf{36.3\%}$ ！命中率只提高2个百分点，性能提升超三分之一。这说明：**失效率减半（5%→3%），平均访问时间可能大幅下降。**

SHARPEN YOUR PENCIL

某32位计算机，Cache容量64KB，块大小16字节，采用4路组相联映射。请计算：

1. Cache共有多少个块？
2. Cache共有多少个组？
3. 主存地址格式（Tag/Index/Offset各多少位）？

SHARPEN YOUR PENCIL SOLUTION

- 地址： | Tag (18位) | Index (10位) | Offset (4位) |

物理页号	V	D	保护位	...
------	---	---	-----	-----

(PPN) 有效 脏位 (读/写/执行)

快表TLB (Translation Lookaside Buffer)：页表在内存中，每次访存要查页表需要多一次内存访问。TLB是页表的Cache，存在CPU内部，通常全相联，几十到几百项。TLB命中率极高 (>99%)，因为程序有很强的空间局部性——一次可能访问同页内连续的多个地址。

地址转换流程：

1. 查TLB (用虚拟页号)
2. TLB命中 → 直接得到物理页号，拼接页内偏移 → 物理地址
3. TLB失效 → 去内存查页表 (可能再引发缺页中断)

WATCH IT!

虚拟地址到物理地址的转换在Cache访问之前还是之后？这取决于Cache是**物理寻址**还是**虚拟寻址**。现代处理器多数是**虚拟索引+物理标记 (VIPT)**：Index用虚拟地址 (可以和TLB并行)，Tag比较用物理地址 (保证正确性)。

段式与段页式

段式虚拟存储器：不是按固定大小的页划分，而是按程序的逻辑段 (代码段、数据段、堆栈段等)。每段大小可变。段描述符里有段基址和段界限。

段页式：先分段，段内再分页。一个虚拟地址 = 段号 + 段内页号 + 页内偏移。虽然灵活，但地址转换要经过段表和页表两次查表，太慢。现代系统大多直接用页式。

页面替换算法

当缺页中断发生、物理内存已满时，需要踢走一页。

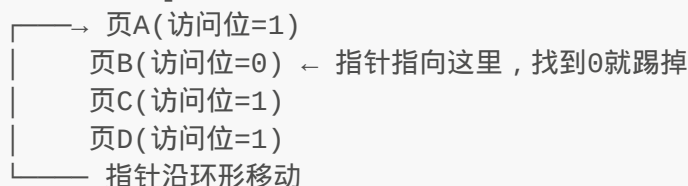
OPT (最佳置换算法)：踢未来最长时间不会被访问的那一页。理论上最优，但无法实现 (因为不知道未来的访问模式)。用于理论分析——和其他算法比"差多少"。

FIFO：踢最早进来的那页。可能把常用页踢了 (Belady异常：分配更多物理页框反而缺页次数增加)。

LRU：踢最近最久未使用的页。性能接近OPT，但每次内存访问都要更新时间戳，开销大。

Clock (时钟置换算法/NRU)：LRU的近似。给每页一个**访问位**，被访问时置1。置换时，指针循环扫描，遇到访问位=0就踢它。遇到访问位=1就置0，给第二次机会。

[Diagram: Clock算法]



GEEK NOTE

Belady异常：FIFO算法下，增加可用物理页框数反而导致缺页率升高。这在LRU等栈算法中不会发生——栈算法保证 $n+1$ 个页框时的缓存内容包含 n 个页框时的所有页（子集性质）。

主存储器：地基打牢了

RAM和ROM

RAM（随机存取存储器）：可读可写，断电丢数据。

- **SRAM**：6个晶体管存1位。快（几纳秒），贵，功耗低。用于Cache。
- **DRAM**：1个晶体管+1个电容存1位。慢（几十纳秒），便宜，但要不断刷新（充电容器）。用于主存。

ROM（只读存储器）：断电不丢数据。

- **PROM**：出厂空白，用户可一次编程
- **EPROM**：紫外线擦除，可多次编程
- **EEPROM**：电擦除，可多次编程（U盘、SSD用的Flash就是EEPROM的变种）

存储器容量扩展

位扩展：每个芯片存1位，用8片组成一个8位的存储"字"。**字扩展**：每个芯片存完整字，用多片增加总字数。通过高位地址+片选信号来实现。**字位扩展**：以上两者结合。

多体交叉存储器

把内存分成多个**存储体（Bank）**，地址交错分配到不同体。访问连续地址时，不同体可以并行工作（流水线访问）。

- **低位交叉**：地址低位选体，高位选体内地址。连续地址分到不同体——适合流水线，一次启动可连续出多个数据。

SHARPEN YOUR PENCIL

某主存容量 $64K \times 8$ 位，现有存储芯片规格为 $16K \times 4$ 位。请设计存储器扩展方案：

1. 需要几片芯片？
2. 位扩展需要几片？字扩展需要几组？
3. 地址线如何分配（哪些位选片，哪些位选片内地址）？

SHARPEN YOUR PENCIL SOLUTION

1. 总容量 $= 64K \times 8 = 512K$ bits，每片 $= 16K \times 4 = 64K$ bits 片数 $= 512K / 64K = 8$ 片
2. 位扩展：每片4位 → 需2片并成8位（位扩展2片一组） 字扩展： $64K / 16K = 4$ 组 总片数 $= 2 \times 4 = 8$ 片 ✓
3. 地址分配（16位地址 $A_{15} \sim A_0$ ）：片内地址： $16K = 2^{14}$ ，需14位（ $A_{13} \sim A_0$ ） 片选（选组）： $64K / 16K = 4$ 组，需2位（ $A_{15} \sim A_{14}$ ） 高位 A_{15}, A_{14} 经2-4译码器产生4个片选信号，选择4组中的一组
 - **高位交叉**：地址高位选体，低位选体内地址。连续地址在同一体——对顺序访问没有优势。

存储体个数 m ，每个体有自己的地址寄存器MAR和数据寄存器MDR等，可以独立工作。

双端口存储器

同一个存储器同时连接两条总线（如CPU和DMA控制器），同时接受两方的读写请求。内部有仲裁逻辑，当两个端口同时访问同一地址时，一个等待。

BRAIN POWER

某32位计算机，虚拟地址空间4GB，页大小4KB，物理内存1GB。请计算：

- (1) 总共有多少个虚拟页？
- (2) 总共有多少个物理页框？
- (3) 如果采用单级页表，每个页表项4字节，页表本身占多大内存？

更进一步想：单级页表对于4GB虚拟地址空间来说，浪费吗？为什么现代操作系统使用多级页表？

BRAIN POWER HINT

(1) 虚拟页数 = $4\text{GB}/4\text{KB} = 2^{32}/2^{12} = 2^{20} = 1,048,576$ 个（约100万页） (2) 物理页框数 = $1\text{GB}/4\text{KB} = 2^{30}/2^{12} = 2^{18} = 262,144$ 个 (3) 单级页表大小 = $2^{20} \times 4\text{B} = 4\text{MB}$ 。注意：这4MB对每个进程都是一份！如果有100个进程，就是400MB。但实际多数页是空的（进程只用了一小部分虚拟空间），4MB大部分被浪费。多级页表只为实际使用的虚拟地址区域分配页表项。

BULLET POINTS（第5章）

- **存储层次**：寄存器 → Cache → 主存 → 辅存。利用**时间局部性**和**空间局部性**原理。
- **Cache映射**：直接映射（快但冲突多）、全相联（灵活但贵）、**组相联**（折中最优）。
- **Cache替换算法**：**LRU**命中率最高，但复杂。随机法简单。FIFO有Belady异常风险。
- **写策略**：写直达（简单、Cache/主存一致）vs写回（性能好、需脏位）。
- **平均访问时间** = 命中率 × 命中时间 + 失效率 × 失效代价。失效率对性能影响巨大。
- **虚拟存储器**：每个进程独立虚拟地址空间，MMU+页表完成地址转换。
- **页式**：固定大小页。**TLB**是页表的小型全相联Cache，加速地址转换。
- **段式**：按逻辑段划分，大小可变。**段页式**：先分段再分页，两段查表开销大。
- **页面替换**：OPT（理想）、LRU（接近最优但开销大）、**Clock**（LRU的实用近似）。
- **SRAM**（快、贵，用于Cache）vs **DRAM**（慢、便宜、需刷新，用于主存）。
- **多体交叉**：**低位交叉**使连续地址分布在不同存储体中，支持流水线并行访问。
- **双端口存储器**：同时服务两路请求，内部仲裁冲突。

BE THE COMPUTER

某计算机Cache参数：容量4KB，块大小16B，2路组相联，LRU替换，写回法+写分配。CPU发出以下访存序列（十六进制地址）：

```
0x0100, 0x0104, 0x0200, 0x0108, 0x0300, 0x0100, 0x0204, 0x010C
```

假设Cache初始为空。请追踪每次访问的命中/失效，并计算最终命中率。提示：4KB/16B=256块，2路组相联 → 128组。Index=7位(128组), Offset=4位(16B)。

BE THE COMPUTER SOLUTION

地址分解 ($\text{Index} = (\text{地址}/16) \bmod 128$) :

- 0x0100: 地址/16 = 0x0010 = 16, Index = 16 mod 128 = 16 → 组16, Tag = 0
- 0x0104: 地址/16 = 0x0010 = 16, Index = 16 → 组16, Tag = 0
- 0x0200: 地址/16 = 0x0020 = 32, Index = 32 → 组32, Tag = 0
- 0x0108: 地址/16 = 0x0010 = 16, Index = 16 → 组16, Tag = 0
- 0x0300: 地址/16 = 0x0030 = 48, Index = 48 → 组48, Tag = 0
- 0x0100: 地址/16 = 16, Index = 16 → 组16, Tag = 0
- 0x0204: 地址/16 = 0x0020 = 32, Index = 32 → 组32, Tag = 0
- 0x010C: 地址/16 = 0x0010 = 16, Index = 16 → 组16, Tag = 0

追踪 (每个块跨0x10=16字节范围) :

1. 0x0100: 组16初始空 → 失效(1), 装入块(0x0100~0x010F)
2. 0x0104: 组16, Tag=0, 有效 → 命中(1)
3. 0x0200: 组32初始空 → 失效(2), 装入块(0x0200~0x020F)
4. 0x0108: 组16, Tag=0, 有效 → 命中(2)
5. 0x0300: 组48初始空 → 失效(3), 装入块(0x0300~0x030F)
6. 0x0100: 组16, Tag=0 → 命中(3)
7. 0x0204: 组32, Tag=0 → 命中(4)
8. 0x010C: 组16, Tag=0 → 命中(5)

命中率 = $5/8 = 62.5\%$ 。初始3个冷失效后, 后续全部命中!

第6章. 输入输出系统与总线：CPU的社交网络

小明把一个U盘插进电脑, "哒"一声, 文件管理器弹出来了。他盯着屏幕："CPU怎么知道U盘插进来了？它又是怎样从U盘里把数据读出来的？CPU一直在算东西, 它哪有空管U盘？"

老张："好问题。CPU不是一个人在战斗。它有整个'社交网络'——总线、中断、DMA, 还有各种I/O接口。这就是本章要讲的：**CPU怎么跟外面的世界打交道。**"

I/O系统概述

计算机的I/O系统负责在CPU/内存和外部设备（键盘、鼠标、硬盘、网卡、打印机等）之间传输数据。

I/O接口：CPU不能直接和设备对话——电压、时序、协议全都不一样。I/O接口（或叫I/O控制器）就是"翻译官"和"转接头"。

I/O接口的功能：

1. 数据缓冲（速度匹配——CPU快、设备慢）
2. 信号转换（并行 ↔ 串行、电平转换）
3. 地址译码（识别CPU发来的设备选择信号）
4. 差错检测
5. 提供设备状态（就绪/忙碌/错误）

I/O编址：怎么找到设备？

统一编址（存储器映射I/O）：I/O端口和存储器统一编址，用同一套地址空间。用Load/Store指令操作设备——就像读写内存一样。优点：不需要专门的I/O指令。缺点：I/O占用了一部分存储器地址空间。

独立编址（端口映射I/O）：I/O端口有独立的地址空间，用专门的IN/OUT指令操作。优点：I/O不占内存地址。缺点：需要专门的I/O指令。

你猜x86用哪种？**独立编址**，用IN/OUT指令。ARM/MIPS呢？**统一编址**——它们根本没有I/O专用指令。

数据传送控制：四种姿势

方式一：程序查询（轮询）

CPU一直反复询问设备："好了没？好了没？好了没？"

```
LOOP: IN STATUS_PORT    ; 读状态寄存器
      TEST READY_BIT    ; 检查就绪位
      JZ LOOP           ; 没就绪继续循环
      IN DATA_PORT     ; 就绪了，读数据
```

这种方式CPU完全被绑定在等待上，CPU利用率极低。但在最简单、最低功耗的场景下偶尔使用。

方式二：程序中断

CPU把任务启动后就不再管了，继续做自己的事。设备准备好了，发一个**中断请求**给CPU。CPU放下手头工作（保护现场），处理中断（读取数据），再回来接着干。

```
CPU: "硬盘，给我读这个文件。"  (发出I/O命令)
CPU: "我先去算别的了。"        (继续执行其他程序)
硬盘: (忙活了一阵...)
硬盘: "数据准备好了！哔——"    (发中断信号)
CPU: "来了！" (保护现场 → 中断服务程序 → 恢复现场)
CPU: "好了，数据拿到了。"
```

优点：CPU不用傻等。缺点：中断开销（保护/恢复现场），不适合大规模数据传输（每次中断传输少量数据）。

方式三：DMA（直接存储器访问）

对于大块数据传输（如从磁盘读一个文件），中断方式太慢——每个字节触发一次中断，CPU要往返N次。

DMA控制器直接接管总线，在设备和主存之间搬运数据，**CPU完全不参与**。等整块数据传完了，DMA给CPU发一个中断："全搬完了，你查收。"

```
DMA方式流程：
CPU → DMA控制器: "把磁盘的1000个字节搬到内存0x1000~0x13E7"
CPU: "我去干别的事了。"
```

DMA：（窃取总线周期，直接把数据从磁盘搬到内存）
DMA → CPU："全部搬完了！"（仅一次中断）

周期窃取（Cycle Stealing）：DMA和CPU共享总线，DMA在CPU不使用总线时"偷"一个总线周期来传输数据。对CPU的干扰极小。

DMA的三种类型：

- **单字节DMA**：每次只偷一个周期传一个字节
- **块DMA**：连续偷多个周期传一个块
- **请求DMA**：设备有请求时偷，传完即放

方式四：通道（I/O Processor）

通道是一个**专门的I/O处理器**，有自己的指令集（通道指令/通道程序）。CPU只需要把通道程序的地址告诉通道，通道就能独立完成整个I/O操作——完全不依赖CPU。通道比DMA更智能，能执行复杂的I/O程序。

字节多路通道：连接多台慢速设备，以字节为单位交叉传输。**选择通道**：连接高速设备，以数据块为单位传输，一次独占通道。**数组多路通道**：结合前两者，连接多台高速设备，块传输但允许交叉。

GEEK NOTE

通道概念源自IBM的大型机（System/360）。通道有自己的处理器和指令，可以执行通道程序来处理复杂的I/O操作。在现代系统中，智能I/O处理器（如RAID卡上的处理器、网卡上的卸载引擎）可以看作是通道思想的延续。

中断系统：CPU的"紧急SOS"

中断的基本概念

中断：CPU正在执行程序时，发生了某种事件（I/O完成、定时时间到、除以零、按下键盘...），CPU暂停当前程序，转而处理该事件，处理完再返回。

中断源分类：

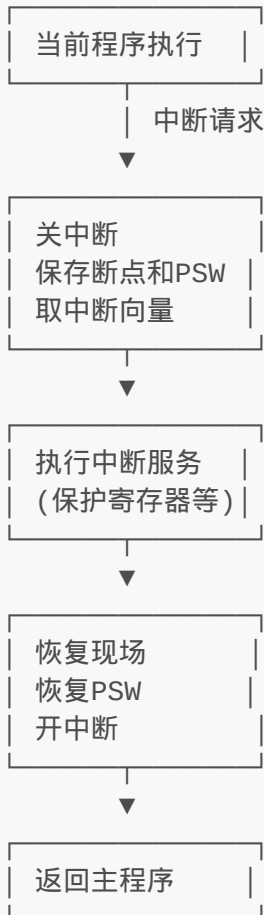
- **外部中断（硬件中断）**：来自CPU外部——I/O设备、定时器、电源掉电等
- **内部中断（异常）**：来自CPU内部——除以零、非法指令、缺页异常等
- **软中断（陷阱）**：程序主动发起——系统调用（INT指令）

中断响应过程

1. CPU在当前指令执行结束后，检查有没有中断请求
2. 如果有且CPU允许响应（中断允许标志IF=1），发出**中断响应信号**
3. **关中断**（防止新的中断干扰现场保存）
4. 保存**断点**和**程序状态字（PSW）** ——通常压入堆栈
5. 根据中断号/向量号，从**中断向量表**中取中断服务程序入口地址
6. 转入中断服务程序执行

中断处理过程

[Diagram: 中断处理流程图]



中断优先级

当多个中断同时到达时，先处理哪个？

排优先级的原则：

1. 故障/异常 > 外部中断（机器错误 > 程序错误 > 定时器 > I/O）
2. 高速设备 > 低速设备（硬盘中断 > 键盘中断）
3. 实时性强的 > 实时性弱的

多重中断与中断屏蔽

多重中断：在中断服务过程中又发生了更高级别的中断，允许嵌套响应。条件是CPU在处理中断时开中断。

中断屏蔽：通过屏蔽字，CPU可以选择性地忽略某些级别或某些来源的中断。用于保护关键代码段不被打断。

每个中断源有一个**屏蔽触发器**：置1时该中断被屏蔽（被忽略）。CPU内部有一个**中断允许触发器（IF）**：为0时屏蔽所有可屏蔽中断。

WATCH IT!

NMI（Non-Maskable Interrupt，不可屏蔽中断）无法被屏蔽。电源掉电、硬件故障等严重事件使用NMI。正常的I/O中断都是可屏蔽的。

SHARPEN YOUR PENCIL

判断下列场景分别属于哪种I/O控制方式，并说明理由：

1. 键盘每按一个键，产生一个信号给CPU
2. 从SSD拷贝一个10GB的电影文件
3. 打印机打印一页内容
4. 温度传感器每100ms报告一次温度

SHARPEN YOUR PENCIL SOLUTION

1. **程序中断方式**。键盘输入数据量极小（一个字节/次），但发生时间不定，中断最合适。
2. **DMA方式**。10GB的数据量太大，中断方式（每次中断仅传少量数据）会让CPU跑瘫痪。DMA直接批量传输，传完才通知CPU。
3. **程序中断方式（或DMA）**。打印一页通常几十KB，中断方式一般够用（每次中断传几个字节）。现代打印机也可以通过USB DMA方式传输。
4. **程序中断方式**。每次数据量极小（几个字节），间隔固定100ms，中断简单高效。

总线系统：数据高速公路

总线的基本概念

总线是一组公共信号线，连接计算机的各个部件，是数据传输的共享通道。

按传输内容分类：

- **数据总线**：传输数据（双向）
- **地址总线**：传输地址信息（单向，CPU → 内存/I/O）
- **控制总线**：传输控制信号（读写、中断、时钟等）

按连接范围分类：

- **片内总线**：CPU芯片内部的总线
- **系统总线**：连接CPU、内存、I/O接口的主板总线
- **外部总线**：连接计算机与外部设备（USB、PCIe等）

总线结构

单总线结构：CPU、内存、所有I/O设备共享一条总线。简单便宜，但所有通信都要争用这根总线，效率低。

双总线结构：一条是CPU和内存之间的"存储总线"（快），另一条是I/O总线。CPU访问内存不用和I/O设备争用总线。但I/O设备不能直接访问内存。

三总线结构：存储总线 + I/O总线 + **DMA总线**（允许I/O设备和内存直接通信，不经过CPU）。现代计算机实际采用的多是这种架构（加上北桥/南桥或芯片组的变种）。

[Diagram: 三总线结构]



总线仲裁：谁先用？

总线上挂了好多设备，但同一时刻只能有一个设备"说话"（发送数据）。谁来当裁判？

集中式仲裁：有一个中央仲裁器决定谁占用总线。

- 1. **链式查询 (Daisy Chaining)：**仲裁器发一个"总线允许"信号沿着菊花链往下传。最近的优先。简单但公平性差（近距离设备可能饿死远距离设备），且单点故障。
- 2. **计数器定时：**仲裁器轮询每个设备号。公平但慢（需要计数器+地址线）。
- 3. **独立请求：**每个设备有自己独立的请求线和允许线。仲裁器内部有复杂的优先级逻辑。最快但信号线最多。

分布式仲裁：没有中央仲裁器。每个设备自己决定是否占用总线。通常通过各设备自己的"仲裁号"在总线上比较——号小的（优先级高的）获胜。

总线通信定时

同步通信：所有设备用同一个时钟信号同步。发送方在时钟边沿放数据，接收方在下一边沿取数据。简单高效，但所有设备必须适应同一个时钟速率。适合短距离、速度接近的总线（如CPU内部总线）。

异步通信：不用统一时钟，而是用**握手信号**（请求/应答）。发送方发出数据后就绪信号，接收方收到后发应答。灵活——可以连接速度差异大的设备，但效率低于同步通信。

半同步通信：结合二者。用统一时钟，但允许慢速设备发"等待"信号让主设备插入等待周期。PCI总线就是半个同步的例子。

老张插了一句："异步通信中，请求和应答的握手就像：'你的包裹到了。'→'好的，我签收。'两边各自按自己的节奏办事。"

常见总线标准

标准	年代	特点
ISA	1980s	8/16位，8.33MHz，已淘汰
EISA	1988	32位，8.33MHz，ISA兼容
PCI	1992	32/64位，33/66MHz，并行总线
PCI Express	2004	串行差分信号，点对点，可扩展
USB	1996	串行，热插拔，随处可见

PCI Express (PCIe) 是现代计算机的主流I/O总线。采用**串行差分信号**，点对点连接——每个设备独占通道（Lane），多个通道可捆绑（x1, x4, x8, x16）。取代了旧的并行共享总线PCI。

GEEK NOTE

PCIe为什么是串行而非并行？表面上看并行更宽，但高频下并行信号存在严重的信号偏斜（Skew）问题——各位到达时间不一致。串行差分信号可以在更高频率下稳定工作。PCIe 3.0单通道速率可达8GT/s（约1GB/s），16通道就是16GB/s。

SHARPEN YOUR PENCIL

某计算机系统有键盘、硬盘、打印机三个I/O设备。键盘中断优先级最高，硬盘次之，打印机最低。当前CPU正在处理硬盘中断服务程序时，键盘和打印机同时发出中断请求。CPU中断允许标志IF=1。请问：

1. CPU会立即响应键盘还是打印机的中断？还是都延后？
2. 如果希望在硬盘中断处理期间不被键盘打断，应该怎么办？
3. 如果打印机中断先到达，键盘中断在打印机中断处理期间到达，CPU会怎么做？

SHARPEN YOUR PENCIL SOLUTION

1. CPU会立即响应键盘中断（优先级高于硬盘），形成**多重中断嵌套**：主程序 → 硬盘中断服务 → 键盘中断服务 → 返回硬盘中断 → 返回主程序 因为IF=1，允许在中断处理中再响应更高级中断。
2. 在硬盘中断服务程序入口处执行**关中断**指令（CLI，IF置0），在服务程序返回前开中断（STI）。这样键盘中断被暂时屏蔽，等硬盘中断处理完再响应。
3. 打印机中断处理期间（IF=1），键盘中断到达。键盘优先级>打印机，CPU暂停打印机中断服务，转去处理键盘中断，键盘处理完再返回继续处理打印机中断。

BRAIN POWER

如果你要设计一个实时控制系统（如无人机飞控），需要保证传感器数据绝不丢失。在中断、DMA和程序查询三种I/O方式中，你会选择哪一种？更进一步想：如果传感器每秒产生100MB数据，你的选择会变吗？

（提示：数据量是决定因素——小数据量中断，大数据量DMA。）

THERE ARE NO DUMB QUESTIONS

Q: 中断和异常有什么区别？A: 中断（Interrupt）通常指外部事件（I/O完成等），发生在指令之间，是异步的。异常（Exception）是CPU在执行指令时发生的问题（除以零、缺页），发生在指令执行过程中，是同步的。广义的“中断”包含了异常。考试中要注意这个区分。

Q: DMA怎么实现“窃取周期”？它和CPU抢总线不会冲突吗？A: DMA只占用CPU不用的总线周期。大多数时间里，CPU在执行内部操作（寄存器—寄存器运算），不使用系统总线。DMA等这些“空隙”插入数据传输。只有当CPU和DMA同时需要总线时，才发生竞争——DMA通常有更高优先权，因为I/O设备的数据缓冲区可能溢出（数据来了不接就丢了）。

Q: 为什么中断向量表重要？A: 它把中断号/类型号映射到中断服务程序的入口地址，使得CPU能快速找到正确的处理程序。在x86实模式下，中断向量表在内存最低的1024字节（0x00000~0x003FF），每个向量占4字节（段地址:偏移地址）。在保护模式下用中断描述符表IDT代替。

BE THE COMPUTER

请模拟一次完整的中断响应和返回过程。假设：

- 主程序执行到地址 0x2100 时，正在执行 ADD 指令
- 此时硬盘发出中断请求 (中断号=0x0D)
- 中断向量表在 0x0000 起始，每个中断向量占4字节
- 向量表中 0x0D×4=0x34 处存放中断服务程序入口地址: 0x3000

请详细描述CPU从收到中断到返回主程序的每一步操作。

BE THE COMPUTER SOLUTION

1. CPU执行完ADD指令（地址0x2100），在下一个时钟周期检查中断请求线，发现有中断
2. 检查IF（中断允许标志）=1，可以响应
3. CPU发出中断响应信号（INTA），从数据总线读取中断号0x0D
4. 关中断（IF置0）
5. 将断点地址压入堆栈： $SS:SP \leftarrow (\text{当前CS:IP的下一条指令地址})$
 - 下一条指令地址 = 0x2100 + ADD指令长度（假设3字节）= 0x2103
 - 把0x2103的段地址(CS)和偏移地址(IP)压栈
6. 将PSW（标志寄存器）压栈
7. 从中断向量表取入口地址：
 - $0x0D \times 4 = 0x34$
 - 从[0x34]取IP = 0x3000的低16位
 - 从[0x36]取CS = 0x3000的高16位
 - 装载 CS:IP = 0x0000:0x3000（实际物理地址 = 0x3000）
8. 开始执行中断服务程序（0x3000处）
 - 保护寄存器（PUSH AX, BX, CX, DX等）
 - 处理I/O数据
 - 向中断控制器发送EOI（End of Interrupt）
 - 恢复寄存器（POP DX, CX, BX, AX）
9. IRET指令：
 - 弹出PSW，恢复标志寄存器
 - 弹出IP和CS，恢复断点地址(0x2103)
 - 开中断（IF恢复为1）
10. 继续执行主程序，从0x2103处开始

BULLET POINTS（第6章）

- **I/O接口**是CPU与外部设备之间的桥梁：数据缓冲、信号转换、地址译码。
- **统一编址**：I/O和内存同一地址空间（ARM/MIPS）。**独立编址**：独立地址空间+专用指令（x86）。
- **程序查询（轮询）**：CPU循环等设备，效率最低。**中断**：设备就绪时发信号，CPU无需等待。
- **DMA**：大批数据传输由DMA控制器直接内存搬运，CPU不参与。传输完成后仅一次中断通知。
- **通道**：专用I/O处理器，有独立通道程序。字节多路、选择、数组多路三种类型。
- **中断响应**：关中断→保存断点和PSW→取中断向量→执行服务程序→恢复→开中断→返回。
- **中断优先级**：多个同时到达时按优先级排序。**中断屏蔽**：有选择地忽略低优先级中断。
- **NMI**（不可屏蔽中断）用于严重事件（掉电等），普通I/O为可屏蔽中断。
- **总线**：数据+地址+控制线。**单总线/双总线/三总线**三种结构。现代多为多级总线。
- **总线仲裁**：链式查询（简单但不公）、计数器定时（轮询公平）、独立请求（快但信号线多）。

- **同步通信**统一时钟，**异步通信**用握手，**半同步**结合二者。
- **PCI Express**取代PCI：串行差分信号、点对点、多通道捆绑，现代I/O总线标配。

第7章. 互连网络与并行处理：1+1>2

老张把两台旧笔记本电脑放在桌上，用一根网线连起来。小明好奇地看着。

"小明，一台电脑算不出来的东西，用十台能不能算出来？"

"那要看怎么连吧？十台电脑，各自算各自的，怎么把结果合起来？"

老张眼睛亮了："这就是**互连网络**和**并行处理系统**要解决的问题。比单机更快，不只靠更快的CPU——而是靠很多CPU一起干活。"

互连网络：CPU之间的"路网"

互连网络的基本概念

互连网络是处理单元之间、处理单元与存储器之间传输数据的物理通道。一个互连网络的好坏，看这几个指标：

- **节点度**：一个节点连接几个邻居。度越大，代价越高，但连接性越好。
- **网络直径**：任意两节点之间最短路径的最大长度。直径越小，通信延迟越低。
- **对分带宽**：把网络切成两半时，横跨切面的链路总带宽。越大则网络越难形成瓶颈。
- **可扩展性**：增加节点时，成本和性能的增长是否线性。

互连函数

互连函数描述网络中节点之间的连接关系。输入是源节点编号，输出是目标节点编号。

常见互连函数（N个节点，编号0到N-1）：

函数名	公式	含义
恒等函数	$I(x) = x$	自己连自己
交换函数	$E(x) = x \oplus 1$	和邻接节点交换（最低位取反）
蝶式函数	$B(x) = x \oplus 2^k$	对应位取反
混洗函数	$S(x) = (x \ll 1) \bmod (N-1)$	左循环移位，完美的全混洗
反位序函数	$R(x) = \text{bit_reverse}(x)$	二进制位全部翻转

例题：N=8时，混洗函数

```
S(0)=0, S(1)=2, S(2)=4, S(3)=6, S(4)=1, S(5)=3, S(6)=5, S(7)=7
```

像洗牌！这正是"混洗"名字的由来。

静态互连网络

拓扑结构固定，节点间的连接是物理的、不变的。

线性阵列：N个节点排成一排，两端节点度=1，内部节点度=2。网络直径=N-1。

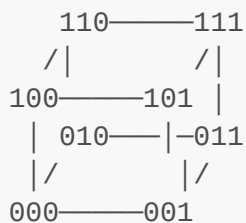
环：首尾相连，每个节点度=2。网络直径= $\lfloor N/2 \rfloor$ 。

2D网格 (Mesh)：N=n×n排列。内部节点度=4，边角节点度=2或3。网络直径=2(n-1)。

树：根、内部节点、叶节点。网络直径=2×树高（经过根来回路程）。

超立方体 (Hypercube)：N=2^k个节点，每个节点有k条边（连接那些编号只有一位不同的节点）。节点度=k，网络直径=k。k维超立方体网络直径仅k，对分带宽N/2。

[Diagram: 3维超立方体]



3维超立方体 (8节点)：每个节点与编号只有1位不同的3个邻居相连。000 连接 001, 010, 100；110 连接 100, 010, 111。网络直径=3。

GEEK NOTE

超立方体在80-90年代的高性能计算中很流行（如Intel iPSC、nCUBE）。它的网络直径 $\log_2 N$ 增长极慢，对分带宽又大，是理论上最优雅的拓扑之一。但物理布线困难（高维下），且扩展时需要加新的维度。现代超算更多用胖树（Fat Tree）或Torus网（环的2D/3D扩展）。

动态互连网络

连接路径可以动态改变——通过开关元件（如交叉开关中的交叉点）调整。

总线：所有节点共享一条通信线。简单，但所有通信串行化，竞争激烈。适合少量节点。

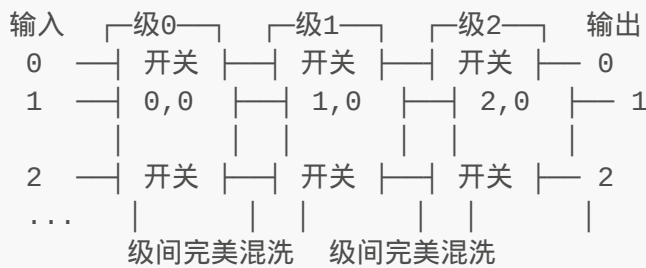
交叉开关 (Crossbar)：N×N的网格，每个交叉点可连通对应行和列。允许N对同时通信而无阻塞！但代价是O(N²)个交叉点，N大了成本爆炸。

多级互连网络 (MIN)：用多级小开关（如2×2开关）组成，典型的如**Omega网络**。

Omega网络：N个输入、N个输出， $\log_2 N$ 级，每级N/2个2×2开关。每个开关只有"直通"和"交叉"两种状态。使用洗牌连接。阻塞型网络——某些输入-输出对之间可能冲突。

Omega网络的自路由特性：目的地址逐级控制开关状态。第i级看目的地址的第(从高到低)i位——0选上端口，1选下端口。

[Diagram: 8输入Omega网络]



消息传递机制

在并行系统中，数据通过消息传递。一个节点发，一个节点收。

路由方式：

- **存储-转发 (Store-and-Forward)**：每个中间节点收完整消息后再发。延迟=跳数×消息长度。
- **虫洞路由 (Wormhole)**：消息分成多个微片 (flit)，第一个微片 (含头部) 在整个路径上"挖洞"，后续微片流水线跟进。延迟接近单跳延迟+消息长度 (不受跳数大影响)。现代互连网络多用虫洞。
- **虚切通路路由 (Virtual Cut-Through)**：混合——目标节点有缓冲时直接切通，否则整包存储转发。

并行处理系统：人多力量大？

并行处理基本概念

并行处理需要多台处理机同时合作解决一个问题。关键挑战不是硬件——而是**任务划分、负载均衡、通信开销和同步**。

Granularity (粒度)：

- 粗粒度：每个处理机干一件大事，偶尔通信。通信开销占比小。
- 细粒度：频繁通信，每个操作都要同步。通信开销可能压倒计算。

超标量与超流水线

这两个概念很多人搞混：

超标量 (Superscalar)：每周期可以发射执行多条指令 (多套功能单元)。横向扩展。

超流水线 (Superpipeline)：把流水线切成更多小段 (更深流水线)，使得时钟频率更高。纵向拉长。

它们可以结合——超标量+超流水线 = 超标量超流水线处理机。每个时钟周期发射多条指令，且流水深度很大。

对称多处理机 (SMP)

SMP：多个相同 (对称) 的处理器通过总线或交叉开关连接到**共享内存**。任何一个处理器可以运行操作系统内核和执行任何进程。

特点：

- 所有CPU地位平等 (对称)
- 共享同一物理内存

- 通过共享内存通信（写入一个地址，另一个CPU读取）
- 必须有Cache一致性机制

你的4核笔记本电脑就是一个小SMP系统！

大规模并行处理机（MPP）

MPP：成百上千个处理节点，每个节点有自己独立的内存。节点间通过高速互连网络通信。用**消息传递**（而非共享内存）通信。

典型结构：每个节点 = CPU + 本地内存 + 网卡。通过高性能网络（如InfiniBand）互连。

编程模型通常是MPI（Message Passing Interface）——你写程序时要显式地发"消息"给其他节点。

机群系统（Cluster）

Cluster = 用普通商用计算机（通常就是标准的PC/服务器）通过以太网/InfiniBand连接，运行集群管理软件。和MPP的核心区别：Cluster用**商用现货（COTS）**组件，而MPP通常用专门设计的硬件。

Cluster性价比极高（可以随时加节点），是当今超级计算机的主流形式。Top500超算榜上绝大多数都是Cluster。

Cache一致性

这个问题的产生很自然：SMP中每个CPU有自己的Cache。CPU-A把变量X改了，存在自己的Cache里。CPU-B也想读X——它读自己Cache里的旧值？还是去主存读最新的？

这就是Cache一致性问题！

两种解决方案：

1. 监听协议（Snooping Protocol）

所有Cache连接在同一总线上。每个Cache控制器"偷听"总线上的地址。当A要写X时，它在总线上广播"我要写X了"。B的Cache控制器听到了，发现"我有X！"，就把自己的副本**无效化（Invalidate）**或**更新（Update）**。

MESI协议是最经典的监听协议。每个Cache行有四种状态：

- **M（Modified，已修改）**：只在本Cache中、已被修改、和主存不一致
- **E（Exclusive，独占）**：只在本Cache中、和主存一致、未修改
- **S（Shared，共享）**：可能在多个Cache中、和主存一致
- **I（Invalid，无效）**：本行数据无效，不可用

状态转移由"本CPU读/写"和"总线上检测到其他CPU读/写"两类事件触发。

MESI状态转移简图：

读命中（I→S或E） | 写命中（M→M） | 总线监听到其他CPU读（M→S）；写（M→I）

2. 基于目录协议（Directory-based Protocol）

大数据量（很多CPU）时，总线广播不再可行（总线负载爆炸）。换用**分布式目录**：用一个集中或分布的"目录"记录每个内存块被哪些Cache持有、状态如何。

CPU要写X时，向目录发请求。目录查：谁有X的副本？向持有者们发**无效化消息**。不用广播，只精确发送。

基于目录协议适合大规模多处理机系统（MPP、大型NUMA服务器）。

WATCH IT!

监听协议要求所有Cache通过总线连接，可扩展性差（总线竞争随CPU数增长）。基于目录协议通过点对点消息实现，可扩展性好，但目录本身是瓶颈且访问目录增加延迟。现代Intel Xeon使用监听协议（小规模），AMD EPYC使用目录协议（大规模）。

向量处理机：大口吃数据

向量处理概念

标量处理：一次一个数。加法器把 $A[0]+B[0]$ 算出得 $C[0]$ ，再来 $A[1]+B[1]$ ，循环N次。

向量处理：一条指令，同时操作整个数组。**向量加法指令**: $C[0:63] = A[0:63] + B[0:63]$ ，一条指令完成64个元素的加法——流水线式地连续64个结果输出。

向量处理的优势：

1. 一条指令完成大量工作，取指开销极小
2. 数据是连续的——内存可以预取、流水化访问
3. 循环中无数据相关（向量内各元素独立）

向量处理机结构

核心部件：

- **向量寄存器**：存放向量操作数。比如Cray-1有8个64元素的向量寄存器。
- **向量功能单元**：可以流水线处理向量元素（如向量加法器、向量乘法器）
- **标量寄存器**：处理标量部分
- **向量Load/Store单元**：从内存加载整条向量

向量链接技术

链接（Chaining）：前一条向量指令的结果元素一旦算出（不需要等整条向量完成），立刻作为下一条指令的输入。相当于向量指令之间的"旁路"——两条或多条向量指令的流水线形成大流水线！

```
V1 ← 内存（向量Load）
V2 ← V0 + V1（向量加法，输入V0是标量扩展或另一向量）
V3 ← V2 × V3（向量乘法，使用链接——V2的元素一经算出就乘）
```

三条指令链接后：每个时钟周期出一个V3的元素！

链接是向量处理机性能的杀手级技术——如果没有链接，每条向量指令必须等前一条全部完成（暂停N个周期再开始下一条）。

GEEK NOTE

Cray-1（1976年）是向量处理机的里程碑。它有80MHz时钟、8个64元素的向量寄存器、三条全流水线功能单元，支持链接和全流水线操作。峰值性能约160 MFLOPS，当时世界第一。Cray-1的设计哲学至今影响GPU和现代向量扩展（AVX-512、ARM SVE）。

BRAIN POWER

超立方体网络中， $N=2^k$ 个节点。对分带宽公式为 $N/2$ ，网络直径 $=k=\log_2 N$ 。如果增加到 $N=1024$ 个节点（ $k=10$ ），网络直径仅从8增到10——似乎增长很慢。那为什么实际超算不都用超立方体拓扑？

（提示：物理布线——10维超立方体每个节点有10条链路，且连接关系非常不"平面"。在实际机柜和电缆中布设10维超立方体几乎不可能。现代超算用胖树或Torus——在物理3D空间中更好排布。）

SHARPEN YOUR PENCIL

在 $N=8$ 的3维超立方体中：

1. 节点000到节点111的最短路径需要几跳？请找出一条路径。
2. 节点001到节点110的最短路径需要几跳？
3. 该网络的节点度、网络直径、对分带宽分别是多少？

SHARPEN YOUR PENCIL SOLUTION

1. $000 \rightarrow 111$ ：因为超立方体的邻接节点只有1位不同，000需改变3位到达111。一条路径：
 $000 \rightarrow 001 \rightarrow 011 \rightarrow 111$ （3跳） 另一条： $000 \rightarrow 010 \rightarrow 110 \rightarrow 111$ （3跳） 答案：**3跳**
2. $001 \rightarrow 110$ ：001和110有3位不同(位0、位1、位2全不同)，需3跳。一条路径： $001 \rightarrow 000 \rightarrow 010 \rightarrow 110$ （3跳） 答案：**3跳**
3. 3维超立方体($N=8, k=3$)：节点度 $= k = 3$ 网络直径 $= k = 3$ 对分带宽 $= N/2 = 4$ 条链路

BE THE COMPUTER

在一个SMP系统中，CPU-0和CPU-1共享内存，使用MESI监听协议。初始时变量X在内存中的值为100，两个CPU的Cache均为空。请追踪以下操作序列中，变量X在每个CPU的Cache中的状态变化：

1. CPU-0 读取 X
2. CPU-1 读取 X
3. CPU-0 写入 $X=200$
4. CPU-1 写入 $X=300$
5. CPU-0 读取 X

BE THE COMPUTER SOLUTION

初始： $X=100$ 在内存，两个Cache空。

1. CPU-0读X：Cache-0失效 $\rightarrow$ 从内存读 $X=100$ Cache-0: X状态=E（独占干净，只有Cache-0有，与内存一致） Cache-1: 无

2. CPU-1读X：总线监听，Cache-0听到 → 从Snoop得知 Cache-0: X状态=S（共享干净，两个Cache都有）
Cache-1: X状态=S（共享干净，从Cache-0或内存获取X=100）
3. CPU-0写X=200：发无效化消息到总线 Cache-0: X状态=M（已修改，X=200，和内存不一致） Cache-1:
监听到无效化 → X状态=I（无效）
4. CPU-1写X=300：Cache-1中的X已无效(I)，写失效 Cache-1发"读-意图修改"请求到总线 Cache-0监听到 →
写回X=200到内存 → X状态 → I Cache-1: 从内存/Cache-0取X=200 → 修改为300 → X状态=M 内存：X=200
（来自Cache-0的写回）
5. CPU-0读X：Cache-0中X=I（无效），发读请求 Cache-1监听到 → 写回X=300到内存 → X状态 → S Cache-0:
从内存/Cache-1取X=300 → X状态=S 内存：X=300 最终：Cache-0=S(300), Cache-1=S(300), 内存=300，
一致 ✓

SHARPEN YOUR PENCIL

在N=8的Omega网络中：

1. 写出输入节点3到输出节点5的路由路径。
2. 写出输入节点6到输出节点3的路由路径。
3. 这两条路径有冲突吗？

提示：Omega网络自路由：把输出节点号作为路由标记，逐级用对应位控制开关（0=上端口，1=下端口）。

SHARPEN YOUR PENCIL SOLUTION

N=8时，地址3位(0~7)。Omega网络3级($\log_2 8=3$)，各级用目的地址的从高位到低位控制开关。

1. 输入3(011) → 输出5(101) 目的地址101：级0: 最高位=1 → 选下端口 级1: 中位=0 → 选上端口 级2: 最低位=1 → 选下端口

路径：输入3 → 级间完美混洗后 → ... → 输出5 逐级追踪：输入端口011

- 级0：开关由目的位(101的最高位1)控制，下端口
- 经过混洗连线到下一级
- 级1：开关由目的位(101的中位0)控制，上端口
- 级2：开关由目的位(101的最低位1)控制，下端口
- 到达输出端口101(即5) ✓

2. 输入6(110) → 输出3(011) 目的地址011：级0: 最高位=0 → 上端口 级1: 中位=1 → 下端口 级2: 最低位=1 → 下端口

路径追踪：类似上面过程，到达输出端口011(即3) ✓

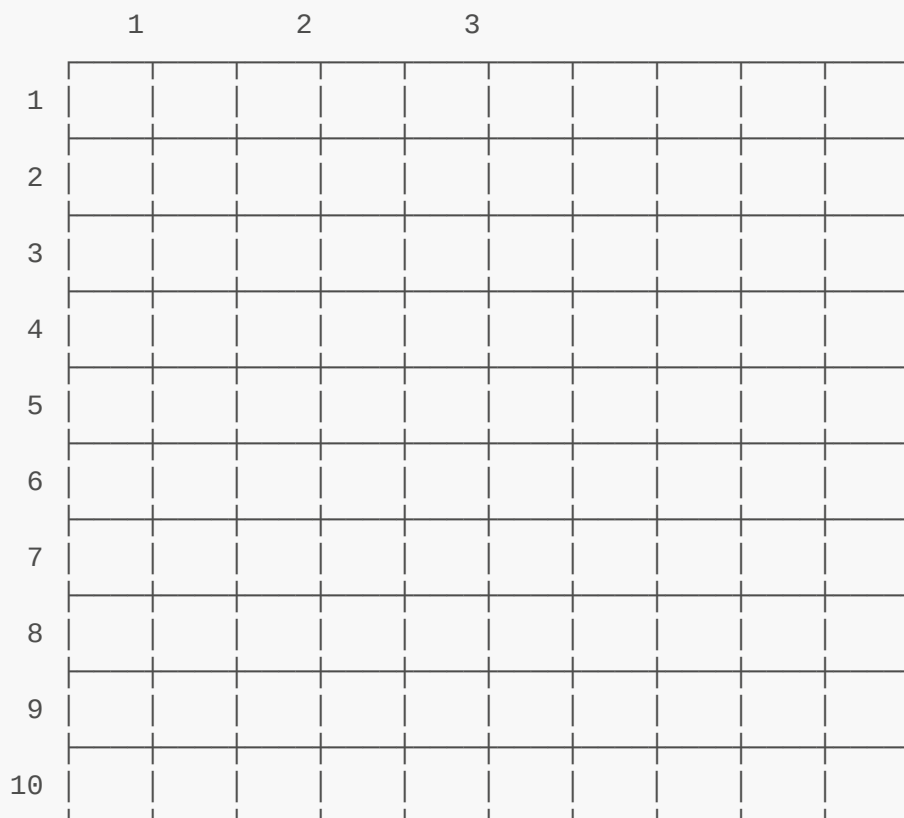
3. 检查冲突：两条路径经过的交换机：路径1: 第0级某个开关 → 第1级某个开关 → 第2级某个开关 路径2: 第0级另一个开关 → 第1级另一个开关 → 第2级另一开关

由于输入端口不同(3 vs 6)，且目的不同(5 vs 3)，经过不同路径。 **没有冲突**。但如果两个输入要求驶向同一个输出端口——那就有冲突了！Omega网络是阻塞网络。

BULLET POINTS（第7章）

- ## 第二篇大结局：计算机系统结构纵横字谜

计算机系统结构 Crossword Puzzle



横行 (Across)

1. Flynn分类法中，单指令流多数据流的缩写（4个字母）

2. 冯·诺依曼结构的核心思想：____程序（存储）
3. IEEE 754浮点数中，表示"非数字"的缩写（3个字母）
4. 替换算法中，踢最早进入的那个块的算法缩写（4个字母）
5. Cache写策略之一，写Cache的同时也写主存：写____（4个汉字）
6. MESI协议中，表示"已修改"的状态（8个字母）

纵行 (Down)

2. 流水线中，不等结果写入寄存器直接转发给下一条指令使用的技术：____技术（旁路）
3. 互连网络中，节点间通过多级2×2开关组成的网络，典型的叫____网络（5个字母）
4. 页面替换算法中，利用访问位环形扫描的算法叫____算法（5个字母）
5. 直接存储器访问的英文缩写（3个字母）

CROSSWORD SOLUTION

	1	2	3						
1	S	I	M	D					
2									
3	存	储				N	A	N	
4									
5		F	I	F	O				
6									
7	直	达							
8									
9	M	O	D	I	F	I	E	D	
10									

本篇终极要点（七章BULLET POINTS合集精华）

- **系统结构层次**：冯·诺依曼五大部件+存储程序。Flynn四分类（SISD/SIMD/MISD/MIMD）。系统结构≠组成≠实现。
- **Amdahl定律**：加速比 = $1/((1-f)+f/S)$ 。串行瓶颈决定理论上限。
- **数据表示**：补码是王者（统一加减、无负零）。IEEE 754：S/E/M，阶码移码，特殊值NAN/∞。
- **指令系统**：七种寻址方式。CISC(多/变长/微程序) vs RISC(少/定长/LoadStore/硬布线)。
- **运算器**：补码加减→溢出判断(Cs⊕Cp)。Booth乘法。浮点加减五步。ALU核心=加法器+逻辑+移位。
- **流水线**：TP=(n)/((k+n-1)τ)。三相关：结构/数据/控制。旁路、延迟分支、分支预测。
- **超标量/VLIW**：超标量运行时调度多发发射。VLIW编译时打包。EPIC=IA-64折中。

- **存储层次**：局部性原理→Cache/主存/辅存三层。Cache映射（直接/全相联/组相联淘汰）。
- **Cache公式**：平均访问时间= $h \times T_{hit} + (1-h) \times T_{miss}$ 。LRU替换+写回/写直达。
- **虚拟存储**：页式+TLB。OPT/LRU/Clock替换算法。MMU转换虚拟→物理地址。
- **I/O方式**：轮询(CPU等)→中断(设备通知)→DMA(批量自传)→通道(专用I/O CPU)。
- **总线**：数据/地址/控制。仲裁(链式/定时/独立请求)。同步/异步/半同步通信。PCIe串行差分。
- **互连网络**：静态(超立方体)和动态(Omega)。虫洞路由。
- **并行处理**：SMP共享内存+Cache一致性(MESI)。MPP/Cluster分布内存+消息传递。
- **向量处理**：单指令操作整数组。链接技术=向量指令间旁路。

老张合上笔记本，转向小明："系统结构到此告一段落。你现在应该能回答我一开始的问题了——什么是计算机系统结构？"

小明自信地笑了："它是对程序员可见的计算机属性的抽象，包括指令集、数据类型、寻址方式、I/O机制。它上面的层次往下看到的是软件界面，它下面的层次是硬件内部的具体实现。"

"完全正确！下一站，我们将进入计算机网络的世界——当计算机不再是孤岛，它们如何通过网线、无线电波、光纤彼此交谈。你准备好了吗？"

[本篇完]